

INDEX

TOPICS	Page No's
➤ Certificates	
➤ Acknowledgement	
➤ Abstract	
➤ Figures/Tables	
CHAPTER-1: INTRODUCTION	1
CHAPTER-2: LITERATURE SURVEY	2-4
CHAPTER-3: SYSTEM ANALYSIS	
3.1 Existing System	5
3.2 Proposed System	5-6
CHAPTER-4: SYSTEM REQUIREMENTS	
4.1 Functional Requirement	7
4.2 Non-Functional Requirements	7
CHAPTER-5: SYSTEM STUDY	
5.1 Feasibility Study	8
5.2 Feasibility Analysis	8-9
CHAPTER-6: SYSTEM DESIGN	
6.1 SYSTEM ARCHITECTURE	10
6.2 UML DIAGRAMS	11-15
6.2.1 Use Case Diagram	
6.2.2 Class Diagram	
6.2.3 Sequence Diagram	
6.2.4 Collaboration Diagram	
6.2.5 Activity Diagram	
6.2.6 Component Diagram	
6.2.7 Deployment Diagram	

6.2.8 Er Diagram

6.2.9 Data Dictionary

CHAPTER-7: INPUT AND OUTPUT DESIGN

7.1 Input Design 16

7.2 Output Design 17

CHAPTER-8: IMPLEMENTATION

8.1 Modules 18

8.1.1 Module Description 18

CHAPTER-9: SOFTWARE ENVIRONMENT

9.1 Python 19-58

9.2 Source Code 58-66

CHAPTER-10: RESULTS/DISCUSSIONS

10.1 System Testing 67-70

10.1.1 Test Cases 70-73

10.2 Output Screens 73-77

CHAPTER-11: CONCLUSION

11.1 Conclusion 78

11.2 Future Scope 78

CHAPTER-12: REFERENCES 79-80

LIST OF FIGURES

S.NO	TABLES/FIGURES	PAGE NO'S
1	System Architecture	10
2	UML Diagrams	11-15
	2.1 Use Case Diagram	11
	2.2 Class Diagram	11
	2.3 Sequence Diagram	12
	2.4 Collaboration Diagram	12
	2.5 Activity Diagram	13
	2.6 Component Diagram	14
	2.7 Deployment Diagram	14
	2.8 ER Diagram	15
	2.9 Data Sets	15
3	Interpretation and Compilation of Python Source code file	19-58
4	Screenshots	73-77

CHILDREN ADHD DISEASE DETECTION USING POSE ESTIMATION TECHNIQUE

ABSTRACT

Attention Deficit Hyperactivity Disorder (ADHD) is predominantly found in children, and its detection often relies on the analysis of children's pose estimation. Currently, there is no automated technique for ADHD detection, and manual monitoring, while being error-prone and challenging, remains the primary method. To address these challenges, we propose the utilization of a machine learning algorithm, specifically the Support Vector Machine (SVM). The SVM will be trained on datasets containing poses from both normal and abnormal instances in children. Subsequently, the algorithm will analyze poses in new test images or videos, providing predictions on whether a child's pose is normal or indicative of ADHD related abnormalities. This approach aims to automate and enhance the accuracy of ADHD detection, contributing to more efficient and reliable diagnostic processes.

CHAPTER-1

INTRODUCTION

Attention Deficit Hyperactivity Disorder (ADHD) is a prevalent neurodevelopmental disorder primarily observed in children. One of the challenging aspects of ADHD detection is relying on manual monitoring of children's pose estimation, which is error-prone and difficult to execute consistently. This project seeks to address these limitations by proposing a machine learning-based solution. Specifically, the project aims to leverage the Support Vector Machine (SVM) algorithm to automate the detection of ADHD in children based on their pose data. This introduction sets the stage for a novel approach to enhance ADHD detection through the application of machine learning techniques.

Attention Deficit Hyperactivity Disorder (ADHD) is a prevalent neurodevelopmental disorder primarily observed in children, characterized by symptoms like inattention, hyperactivity, and impulsivity. Despite its prevalence, detecting ADHD accurately poses significant challenges, particularly due to the reliance on manual monitoring of children's pose estimation. This method is error-prone and challenging to execute consistently, often leading to misdiagnosis or delayed diagnosis.

To address these limitations, a novel project is proposed, aiming to leverage machine learning techniques for automated ADHD detection. Specifically, the project focuses on utilizing the Support Vector Machine (SVM) algorithm, renowned for its effectiveness in classification tasks, to analyze pose data and identify potential indicators of ADHD. By training the SVM model on a dataset comprising pose data from both ADHD-diagnosed and neurotypical children, the system can learn to recognize patterns associated with ADHD symptoms.

This innovative approach holds promise for revolutionizing ADHD detection by providing a more objective and reliable method. By automating the analysis of pose data, clinicians can potentially expedite the diagnostic process and improve accuracy, leading to better outcomes for children with ADHD. Moreover, the application of machine learning in this context not only enhances detection but also opens avenues for deeper understanding of the underlying neurological mechanisms contributing to ADHD. Overall, this project represents a significant step towards advancing both the diagnosis and understanding of ADHD through the integration of cutting-edge technology and clinical practice.

CHAPTER-2

LITERATURE SURVEY

TITLE: Investigating ADHD subtypes in children using temporal dynamics of the electroencephalogram (EEG) microstates

AUTHOR: Na Luo; Xiangsheng Luo; Dongren Yao; Vince D. Calhoun; Li Sun; Jing Sui

ABSTRACT: Attention-deficit/hyperactivity disorder (ADHD) is a prevalent neurodevelopmental disorder in children, usually categorized as three predominant subtypes, persistent inattention (ADHD-I), hyperactivity-impulsivity (ADHD-HI) and a combination of both (ADHD-C). Identifying reliable features to distinguish different subtypes is significant for clinical individualized treatment. In this work, we conducted a two-stage electroencephalogram (EEG) microstate analysis on 54 healthy controls and 107 ADHD children, including 54 ADHD-Is and 53 ADHD-Cs, aiming to examine the dynamic temporal alterations in ADHDs compared to healthy controls (HCs), as well as different EEG signatures between ADHD subtypes. Results demonstrated that the dynamics of resting-state EEG microstates, particularly centering on salience (state C) and frontal-parietal network (state D), were significantly aberrant in ADHDs. Specifically, the occurrence and coverage of state C were decreased in ADHDs ($p=0.002$; $p=0.0015$), while the duration and contribution of state D were observably increased ($p=0.0016$; $p=0.0001$) compared to HCs. Moreover, the transition probability between state A and C was significantly decreased ($p=9.85e-7$; $p=2.33e-7$) in ADHDs, but otherwise increased between state B and D ($p=1.02e-7$; $p=1.07e-6$). By contrast, remarkable subtype differences were found primarily on the visual network (state B) between ADHD-Is and ADHD-Cs. Specifically, ADHD-Cs have higher occurrence and coverage of state B than ADHD-Is ($p=9.35e-5$; $p=1.51e-8$), suggesting these patients more impulsively aimed to open their eyes when asked to keep eyes closed during the data collection. In summary, this work carefully leveraged EEG temporal dynamics to investigate the aberrant microstate features in ADHDs and provided a new window to look into the subtle differences between ADHD subtypes, which may help to assist precision diagnosis in future. Clinical Relevance— This work established the use of EEG microstate features to investigate ADHD dysfunction and its subtypes, providing a new window for better diagnosis of ADHD.

TITLE: Human Body Pose Estimation and Applications

AUTHOR: Amrutha K; Prabu P; Joy Paulose

ABSTRACT: Human Pose Estimation is one of the challenging yet broadly researched areas. Pose estimation is required in applications that include human activity detection, fall detection, motion capture in AR/VR, etc. Nevertheless, images and videos are required for every application that captures images using a standard RGB camera, without any external devices. This paper presents a real-time approach for sign language detection and recognition in videos using the Holistic pose estimation method of MediaPipe. This Holistic framework detects the movements of multiple modalities-facial expression, hand gesture and body pose, which is the best for the sign language recognition model. The experiment conducted includes five different signers, signing ten distinct words in a natural background. Two signs, “blank” and “sad,” were best recognized by the model.

TITLE: A Comparative Study of Pose Estimation

AUTHOR: Raghvender Changotra; Shrey Raj Singh; Piyush Soni; Asmi Zutshi; Ayushi Gupta; Tushar Srivastava; Dinesh Vij; Gurjot Si

ABSTRACT: In the past years, the vision of computers has been able to achieve massive momentum that could provide the absolute best of expectations among the many fields and sectors. A research paper on Human Pose Estimation would open new opportunities and help those who researched in finding the best techniques and methods for human pose detection. In this paper, we proposed that by using Open Pose 2D based pose estimation model technique to track and calculate every minute movement and perform analysis in real time. This technique which has been chosen has an amazing advantage of reasoning about pose in a comprehensive manner and has a straightforward but yet efficacious conceptualization which takes advantage of the recent advances in the Computer Vision Sector. We have presented an elaborate factual analysis with performance that is one step ahead based on four scholastic standards of varied real world representations.

TITLE: A Study for ADHD Identification using Eye Movement Data

AUTHOR: Hansol Ko; Bohyun Wang; Joon. S. Lim

ABSTRACT: Attention deficit hyperactivity disorder (ADHD) is a disorder that occurs a lot in childhood, and is a state of distraction, hyperactivity, and impulsiveness due to a continuous lack of attention. We classified ADHD and control using eye movement data by deep learning. Through seven kinds of VR game, we collected eye movement data from subjects. As deep learning algorithm, we used Zoom in Neural Network(ZNN) based on NEWFM. As classification accuracy, 82.1 % and 83.6% were obtained for two groups, respectively.

TITLE: Review on Gait Analysis using Pose Estimation

AUTHOR: Dimple Sethi; Taniya Sharma; Parul Sehrawat; Himanshi; Aditi Mohanty

ABSTRACT: The gait analysis is systematic study that aims to specifically study the human motion. Therefore Gait analysis is successfully performed for performing pose estimation. This paper aims to summarize the research the gait analysis performed for doing human pose estimation, verifying if it can support medical or clinical evaluation. The paper talks about different methods, activity, parameters, limitations and datasets involved while doing pose estimation using gait analysis and methods which were accurate and better in every perspective. Several approaches show success of gait techniques for detecting disorders i.e used for clinical purpose and also used for solving several criminal cases. Further research is, however, the need of the hour into this field.

CHAPTER-3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

Currently, ADHD detection heavily relies on manual monitoring and assessment of children's pose estimation. This approach is inherently prone to errors and poses difficulties in consistently identifying ADHD-related abnormalities. The absence of an automated technique hampers the efficiency and reliability of the diagnostic process. Therefore, there is a crucial need to introduce a more advanced and accurate system that can automate ADHD detection, overcoming the shortcomings of the current manual monitoring methods.

3.1.1 DISADVANTAGES

The existing system for ADHD detection, which heavily relies on manual monitoring and assessment of children's pose estimation, suffers from several disadvantages. Firstly, this approach is inherently prone to errors due to human subjectivity and variability in interpretation, leading to inconsistencies in diagnosis. Additionally, manual monitoring requires significant time and resources, making it impractical for large-scale screening or continuous monitoring. Moreover, the reliance on subjective assessments may overlook subtle or early signs of ADHD, resulting in delayed diagnosis and intervention. Furthermore, the absence of an automated technique not only hampers the efficiency and reliability of the diagnostic process but also limits access to specialized care, especially in underserved communities. Overall, the limitations of the current system underscore the crucial need for introducing a more advanced and accurate automated system to improve ADHD detection and intervention.

3.2 PROPOSED SYSTEM

The proposed system involves the implementation of a machine learning algorithm, specifically the Support Vector Machine (SVM). This algorithm will undergo training using datasets comprising both normal and abnormal children's poses. Once trained, the SVM algorithm will be capable of analyzing new test images or videos to predict whether a child's pose is normal

or indicative of ADHD-related abnormalities. By integrating machine learning into the detection process, the proposed system aims to offer a more efficient, consistent, and automated solution for ADHD detection, contributing to early intervention and improved accuracy in diagnoses.

3.2.1 ADVANTAGES

The proposed system, integrating a machine learning algorithm such as the Support Vector Machine (SVM), brings several notable advantages to ADHD detection. Firstly, by utilizing datasets encompassing both typical and atypical children's poses for training, the SVM algorithm can discern patterns indicative of ADHD-related abnormalities with a high degree of accuracy. This enables the system to swiftly analyze new images or videos, providing rapid and reliable assessments of a child's pose. Automation through machine learning streamlines the detection process, reducing reliance on manual assessment and minimizing the risk of human error. The consistency afforded by the algorithm ensures uniform evaluations across cases, enhancing the reliability of diagnoses. Moreover, the system's efficiency facilitates timely interventions, crucial for mitigating the impact of ADHD on a child's development. By offering a more efficient, consistent, and automated solution, the proposed system not only aids in early intervention but also elevates the overall accuracy of ADHD diagnoses, thereby improving outcomes for affected individuals.

CHAPTER-4

SYSTEM REQUIREMENTS

4.1 FUNCTIONAL REQUIREMENTS

- **User:** In this module, the User has to login by using valid user name and password. After login successful he can do some operations

4.2 NON-FUNCTIONAL REQUIREMENTS

4.2.1 HARDWARE REQUIREMENTS

MINIMUM (Required for Execution)		MY SYSTEM (Development)
System	Pentium IV 2.2 GHz	i3 Processor 5 th Gen
Hard Disk	20 Gb	512 Gb
Ram	1 Gb	4 Gb

4.2.2 SOFTWARE REQUIREMENTS:

Operating System		Windows 10/11
Development Software		Python 3.7.0
Programming Language		Python
Integrated Development Environment (IDE)		Python IDE
Design/Modelling		Rational Rose
Framework		TKinter

CHAPTER-5

SYSTEM STUDY

5.1 FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

5.2 FEASIBILITY ANALYSIS

Three key considerations involved in the feasibility analysis are

- **ECONOMICAL FEASIBILITY**
- **TECHNICAL FEASIBILITY**
- **SOCIAL FEASIBILITY**

ECONOMICAL FEASIBILITY

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

TECHNICAL FEASIBILITY

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement,

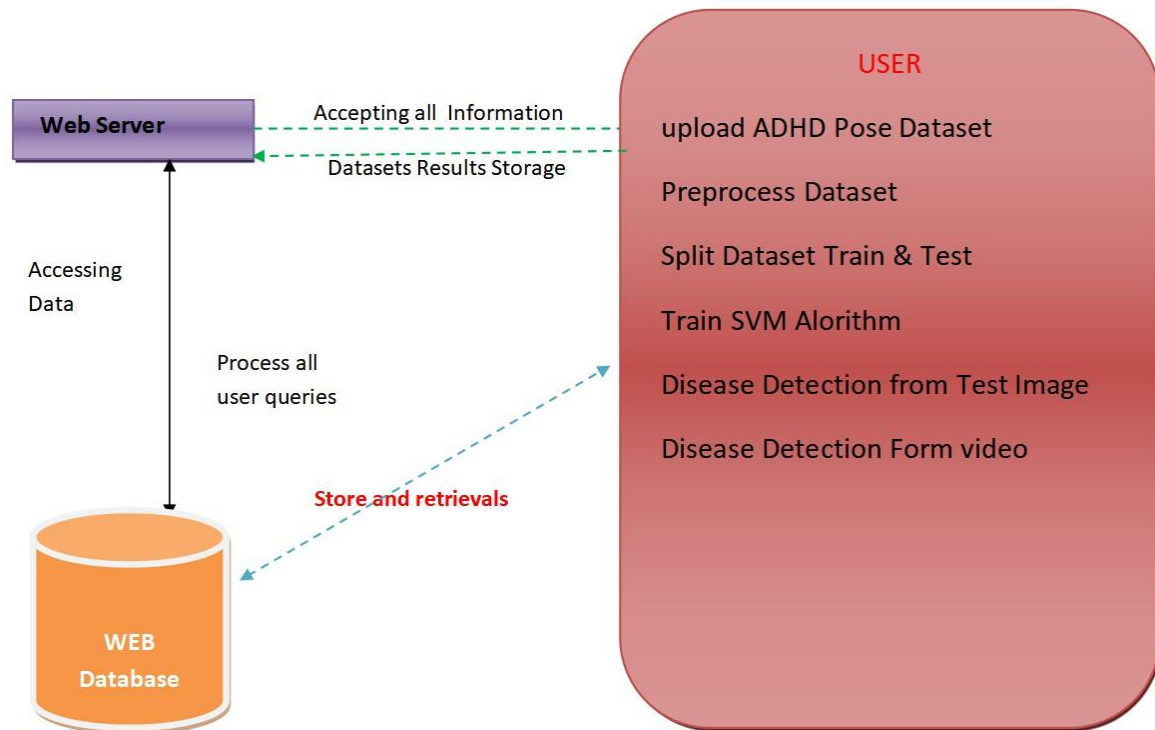
SOCIAL FEASIBILITY

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

CHAPTER-6

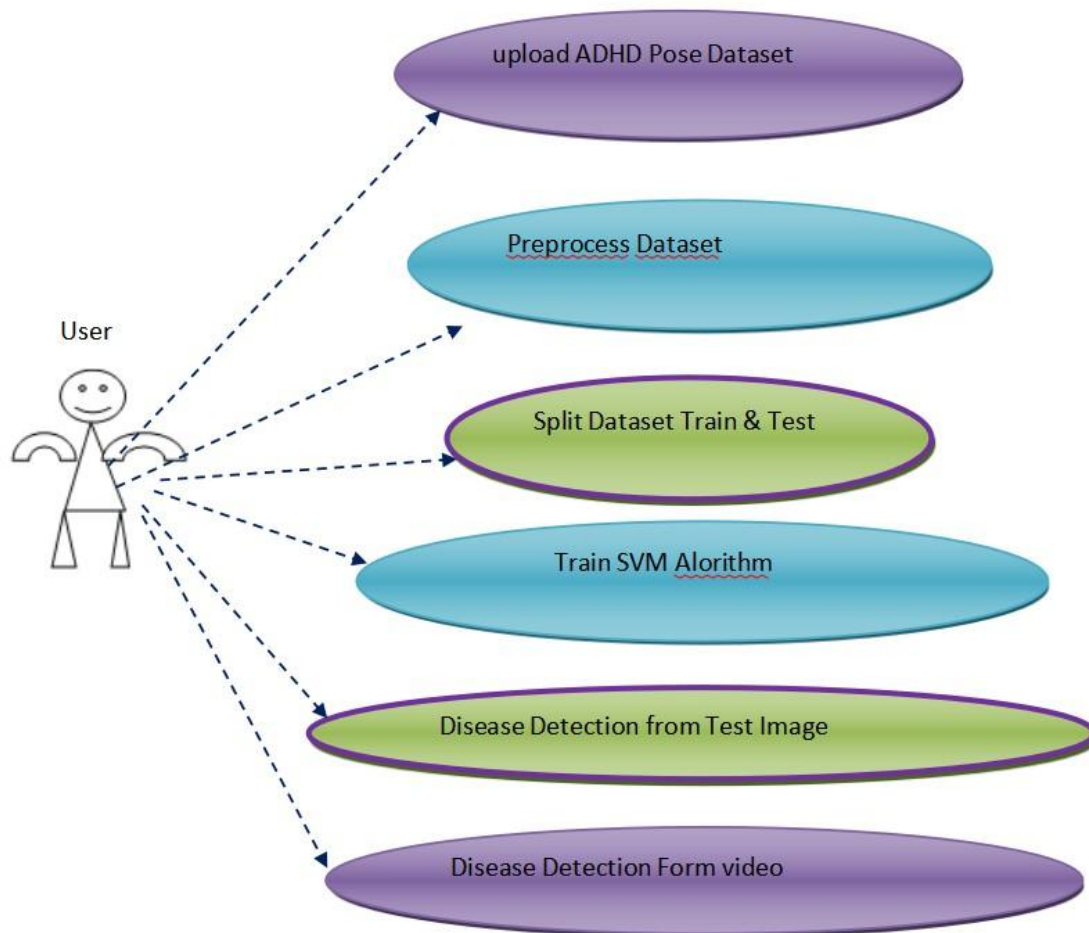
SYSTEM DESIGN

6.1 SYSTEM ARCHITECTURE

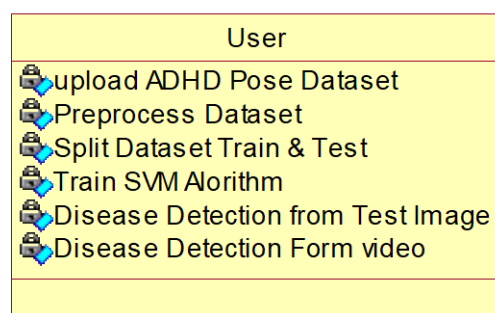


6.2 UML DIAGRAMS

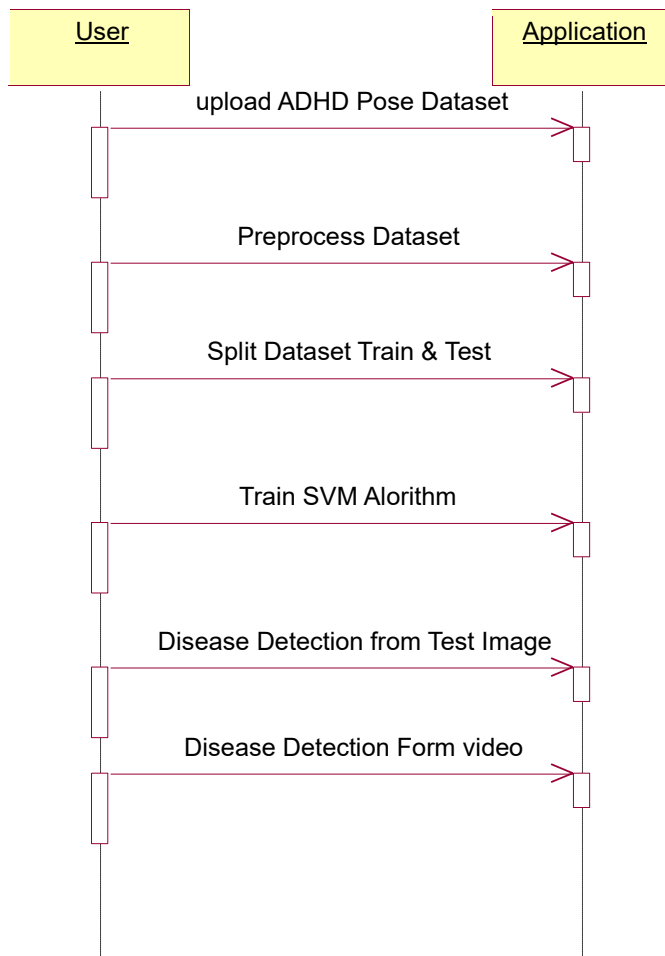
6.2.1 USE CASE DIAGRAM



6.2.2 CLASS DIAGRAM



6.2.3 SEQUENCE DIAGRAM

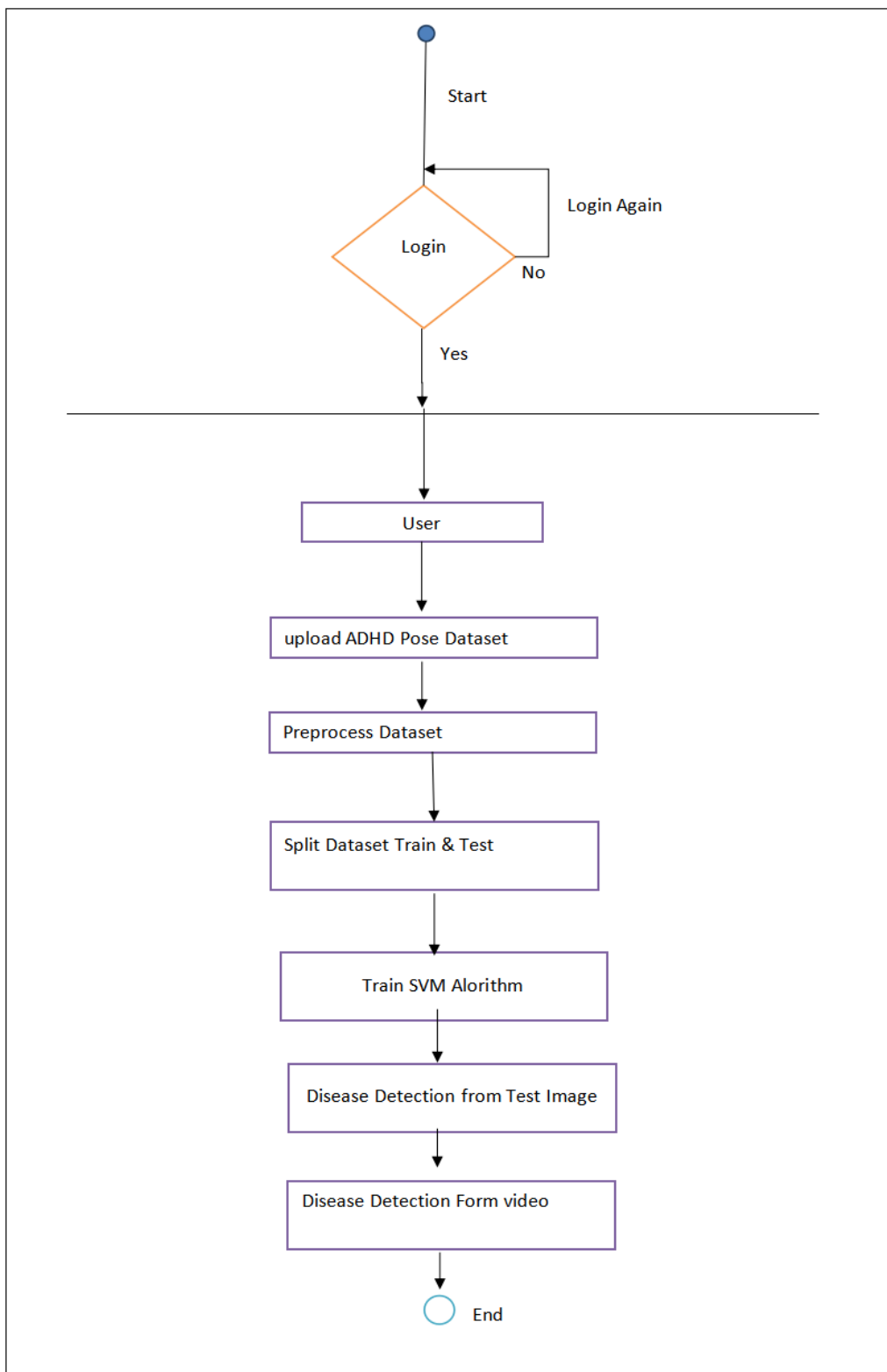


6.2.4 COLLABRATION DIAGRAM

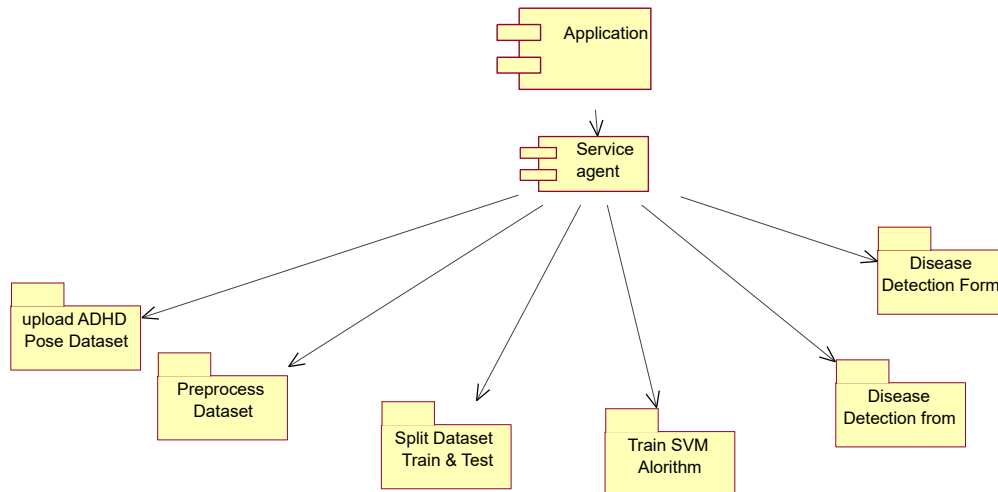
- 1: upload ADHD Pose Dataset
- 2: Preprocess Dataset
- 3: Split Dataset Train & Test
- 4: Train SVM Alorithm
- 5: Disease Detection from Test Image
- 6: Disease Detection Form video



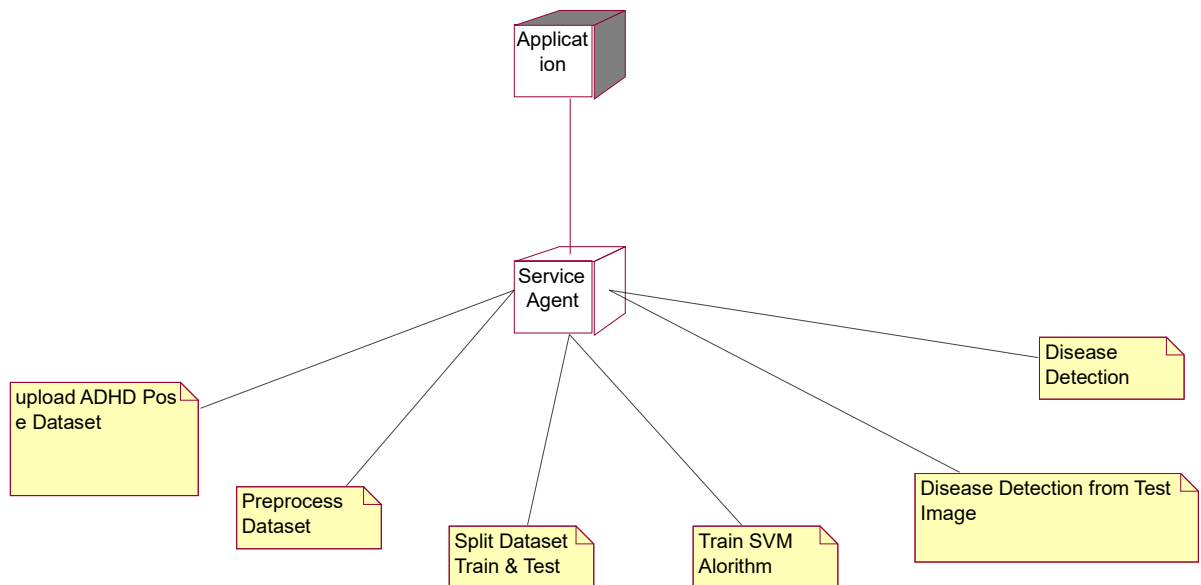
6.2.5 ACTIVITY DIAGRAM



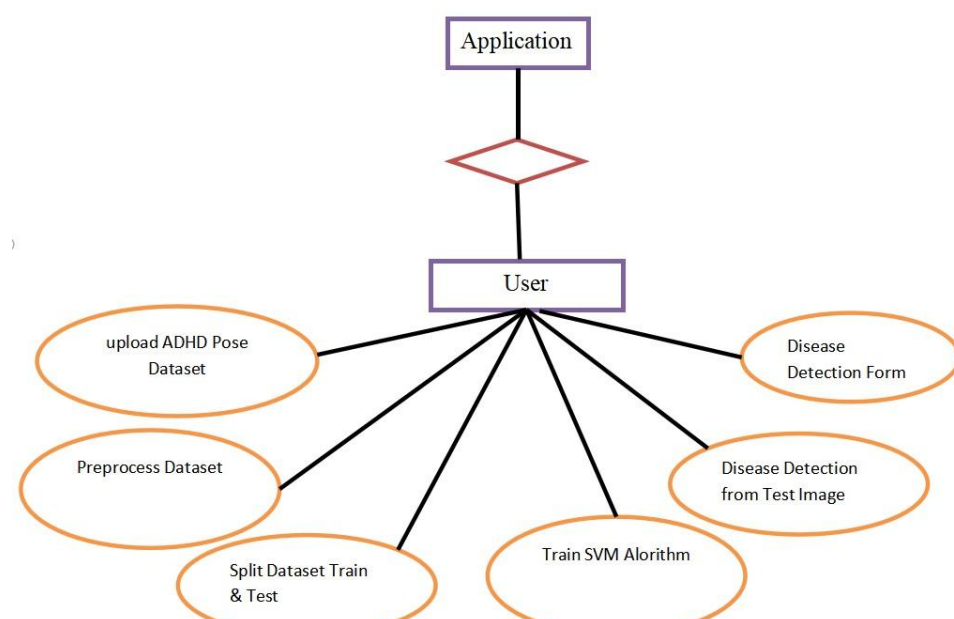
6.2.6 COMPONENT DIAGRAM



6.2.7 DEPLOYMENT DIAGRAM



6.2.8 ER DIAGRAM



6.2.9 DATA SETS

[illegible]

CHAPTER-7

INPUT AND OUTPUT DESIGN

7.1 INPUT DESIGN

The input design is the link between the information system and the user. It comprises the developing specification and procedures for data preparation and those steps are necessary to put transaction data in to a usable form for processing can be achieved by inspecting the computer to read data from a written or printed document or it can occur by having people keying the data directly into the system. The design of input focuses on controlling the amount of input required, controlling the errors, avoiding delay, avoiding extra steps and keeping the process simple. The input is designed in such a way so that it provides security and ease of use with retaining the privacy. Input Design considered the following things:

- What data should be given as input?
- How the data should be arranged or coded?
- The dialog to guide the operating personnel in providing input.
- Methods for preparing input validations and steps to follow when error occur.

7.1.1 OBJECTIVES

1. Input Design is the process of converting a user-oriented description of the input into a computer-based system. This design is important to avoid errors in the data input process and show the correct direction to the management for getting correct information from the computerized system.

2. It is achieved by creating user-friendly screens for the data entry to handle large volume of data. The goal of designing input is to make data entry easier and to be free from errors. The data entry screen is designed in such a way that all the data manipulates can be performed. It also provides record viewing facilities.

3. When the data is entered it will check for its validity. Data can be entered with the help of screens. Appropriate messages are provided as when needed so that the user will not be in maize of instant. Thus the objective of input design is to create an input layout that is easy to follow.

7.2 OUTPUT DESIGN

A quality output is one, which meets the requirements of the end user and presents the information clearly. In any system results of processing are communicated to the users and to other system through outputs. In output design it is determined how the information is to be displaced for immediate need and also the hard copy output. It is the most important and direct source information to the user. Efficient and intelligent output design improves the system's relationship to help user decision-making.

1. Designing computer output should proceed in an organized, well thought out manner; the right output must be developed while ensuring that each output element is designed so that people will find the system can use easily and effectively. When analysis design computer output, they should Identify the specific output that is needed to meet the requirements.

2. Select methods for presenting information.

3. Create document, report, or other formats that contain information produced by the system.

The output form of an information system should accomplish one or more of the following objectives.

- Convey information about past activities, current status or projections of the
- Future.
- Signal important events, opportunities, problems, or warnings.
- Trigger an action.
- Confirm an action.

CHAPTER-8

IMPLEMENTATION

8.1 MODULES

- User

8.1.1 MODULE DESCRIPTION

USER

In this module, there are n numbers of users are present. User should register before doing any operations. Once user registers, their details will be stored to the database. After registration successful, he has to login by using authorized user name and password. Once Login is successful user will do some operations like REGISTER AND LOGIN, Predict Agricultural _text _classification, VIEW YOUR PROFILE.

VIEW AND AUTHORIZE USERS

In this module, the admin can view the list of users who all registered. In this, the admin can view the user's details such as, user name, email, address and admin authorize the users.

CHAPTER-9

SOFTWARE ENVIRONMENT

9.1 PYTHON

Python is a **high-level, interpreted, interactive and object-oriented scripting language**. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

- **Python is Interpreted:** Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- **Python is Interactive:** You can actually sit at a Python prompt and interact with the interpreter directly to write your programs.
- **Python is Object-Oriented:** Python supports Object-Oriented style or technique of programming that encapsulates code within objects.
- **Python is a Beginner's Language:** Python is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.

History of Python

Python was developed by Guido van Rossum in the late eighties and early nineties at the National Research Institute for Mathematics and Computer Science in the Netherlands.

Python is derived from many other languages, including ABC, Modula-3, C, C++, Algol-68, SmallTalk, and Unix shell and other scripting languages.

Python is copyrighted. Like Perl, Python source code is now available under the GNU General Public License (GPL).

Python is now maintained by a core development team at the institute, although Guido van Rossum still holds a vital role in directing its progress.

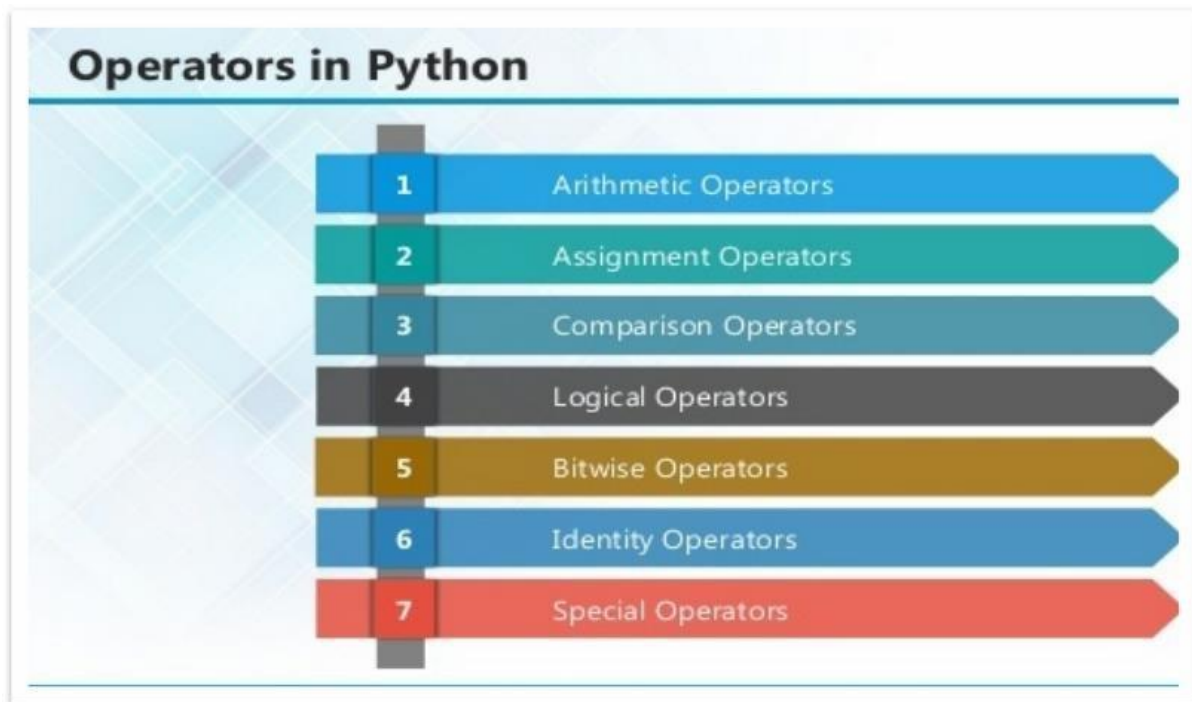
Python Features

Python's features include:

- **Easy-to-learn:** Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.
- **Easy-to-read:** Python code is more clearly defined and visible to the eyes.
- **Easy-to-maintain:** Python's source code is fairly easy-to-maintain.
- **A broad standard library:** Python's bulk of the library is very portable and crossplatform compatible on UNIX, Windows, and Macintosh.
- **Interactive Mode:** Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.
- **Portable:** Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- **Extendable:** You can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.
- **Databases:** Python provides interfaces to all major commercial databases.
- **GUI Programming:** Python supports GUI applications that can be created and ported to many system calls, libraries and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.
- **Scalable:** Python provides a better structure and support for large programs than shell scripting.

Python has a big list of good features:

- It supports functional and structured programming methods as well as OOP.
- It can be used as a scripting language or can be compiled to byte-code for building large applications.
- It provides very high-level dynamic data types and supports dynamic type checking.
- IT supports automatic garbage collection.
- It can be easily integrated with C, C++, COM, ActiveX, CORBA, and Java.



ARITHMETIC OPERATORS

Operator	Description	Example
+ Addition	Adds values on either side of the operator.	$a + b = 30$
- Subtraction	Subtracts right hand operand from left hand operand.	$a - b = -10$
* Multiplication	Multiplies values on either side of the operator	$a * b = 200$

/ Division	Divides left hand operand by right hand operand	$b / a = 2$
% Modulus	Divides left hand operand by right hand operand and returns remainder	$b \% a = 0$
** Exponent	Performs exponential (power) calculation on operators	$a^{**}b = 10$ to the power 20
//	Floor Division - The division of operands where the result is the quotient in which the digits after the decimal point are removed. But if one of the operands is negative, the result is floored, i.e., rounded away from zero (towards negative infinity):	$9//2 = 4$ and $9.0//2.0 = 4.0$, - $11//3 = -$
		4, - $11.0//3 =$ -4.0

ASSIGNMENT OPERATOR

Operator	Description	Example
=	Assigns values from right side operands to left side operand	$c = a + b$ assigns value of a + b into c

+= Add AND	It adds right operand to the left operand and assign the result to left operand	$c += a$ is equivalent to $c = c + a$
-= Subtract AND	It subtracts right operand from the left operand and assign the result to left operand	$c -= a$ is equivalent to $c = c - a$
*= Multiply AND	It multiplies right operand with the left operand and assign the result to left operand	$c *= a$ is equivalent to $c = c * a$
/= Divide AND	It divides left operand with the right operand and assign the result to left operand	$c /= a$ is equivalent to $c = c / a$

%= Modulus AND	It takes modulus using two operands and assign the result to left operand	$c \% a$ is equivalent to $c = c \% a$
**= Exponent AND	Performs exponential (power) calculation on operators and assign value to the left operand	$c ** a$ is equivalent to $c = c ** a$
//= Floor Division	It performs floor division on operators and assign value to the left operand	$c // a$ is equivalent to $c = c // a$

IDENTITY OPERATOR

Operator	Description	Example
is	Evaluates to true if the variables on either side of the operator point to the same object and false otherwise.	x is y, here is results in 1 if id(x) equals id(y).
is not	Evaluates to false if the variables on either side of the operator point to the same object and true otherwise.	x is not y, here is not results in 1 if id(x) is not equal to id(y)

COMPARISON OPERATOR

Operator	Description	Example
& Binary AND	Operator copies a bit to the result if it exists in both operands	(a & b) (means 0000 1100)
Binary OR	It copies a bit if it exists in either operand.	(a b) = 61 (means 0011 1101)
^ Binary XOR	It copies the bit if it is set in one operand but not both.	(a ^ b) = 49 (means 0011 0001)

~ Binary Ones Complement	It is unary and has the effect of 'flipping' bits.	(~a) = -61 (means 1100 0011 in 2's complement form due to a signed binary number.
<< Binary Left Shift	The left operands value is moved left by the number of bits specified by the right operand.	a << 2 = 240 (means 1111 0000)
>> Binary Right Shift	The left operands value is moved right by the number of bits specified by the right operand.	a >> 2 = 15 (means 0000 1111)

LOGICAL OPERATOR

Operator	Description	Example
and Logical AND	If both the operands are true then condition becomes true.	(a and b) is true.
or Logical OR	If any of the two operands are non-zero then condition becomes true.	(a or b) is true.
not Logical NOT	Used to reverse the logical state of its operand.	Not(a and b) is false.

MEMBERSHIP OPERATORS

Operator	Description	Example
in	Evaluates to true if it finds a variable in the specified sequence and false otherwise.	x in y, here in results in a 1 if x is a member of sequence y.
not in	Evaluates to true if it does not finds a variable in the specified sequence and false otherwise.	x not in y, here not in results in a 1 if x
		is not a member of sequence y.

PYTHON OPERATORS PRECEDENCE

Operator	Description
**	Exponentiation (raise to the power)
~ + -	Complement, unary plus and minus (method names for the last two are +@ and -@)
* / % //	Multiply, divide, modulo and floor division
+ -	Addition and subtraction

>> <<	Right and left bitwise shift
&	Bitwise 'AND'
^	Bitwise exclusive 'OR' and regular 'OR'
<= < > >=	Comparison operators
<> == !=	Equality operators
= %= /= //= -= += *= **=	Assignment operators
is is not	Identity operators
in not in	Membership operators
not or and	Logical operators

LIST

The list is a most versatile data type available in Python which can be written as a list of commaseparated values (items) between square brackets. Important thing about a list is that items in a list need not be of the same type.

Creating a list is as simple as putting different comma-separated values between square brackets. For example –


```
list1 = ['physics', 'chemistry', 1997, 2000];  
list2 = [1, 2, 3, 4, 5]; list3 = ["a", "b", "c",  
"d"]
```

BASIC LIST OPERATIONS

Lists respond to the + and * operators much like strings; they mean concatenation and repetition here too, except that the result is a new list, not a string.

Python Expression	Results	Description
len([1, 2, 3])	3	Length
[1, 2, 3] + [4, 5, 6]	[1, 2, 3, 4, 5, 6]	Concatenation
['Hi!'] * 4	['Hi!', 'Hi!', 'Hi!', 'Hi!']	Repetition
3 in [1, 2, 3]	True	Membership
for x in [1, 2, 3]: print x,	1 2 3	Iteration

BUILT-IN LIST FUNCTIONS & METHODS:

Python includes the following list functions –

SN	Function with Description
1	<u>cmp(list1, list2)</u> Compares elements of both lists.
2	<u>len(list)</u> Gives the total length of the list.
3	<u>max(list)</u> Returns item from the list with max value.
4	<u>min(list)</u> Returns item from the list with min value.
5	<u>list(seq)</u> Converts a tuple into list.

Python includes following list methods

SN	Methods with Description
----	--------------------------

1	<u>list.append(obj)</u> Appends object obj to list
2	<u>list.count(obj)</u> Returns count of how many times obj occurs in list
3	<u>list.extend(seq)</u> Appends the contents of seq to list
4	<u>list.index(obj)</u> Returns the lowest index in list that obj appears
5	<u>list.insert(index, obj)</u> Inserts object obj into list at offset index
6	<u>list.pop(obj=list[-1])</u> Removes and returns last object or obj from list
7	<u>list.remove(obj)</u> Removes object obj from list
8	<u>list.reverse()</u> Reverses objects of list in place

9

[list.sort\(\[func\]\)](#)

Sorts objects of list, use compare function if given

TUPLES

A tuple is a sequence of immutable Python objects. Tuples are sequences, just like lists. The differences between tuples and lists are, the tuples cannot be changed unlike lists and tuples use parentheses, whereas lists use square brackets.

Creating a tuple is as simple as putting different comma-separated values. Optionally we can put these comma-separated values between parentheses also. For example –

```
tup1 = ('physics', 'chemistry', 1997, 2000);
tup2 = (1, 2, 3, 4, 5 ); tup3 = "a", "b", "c",
"d";
```

The empty tuple is written as two parentheses containing nothing –

```
tup1 = ();
```

To write a tuple containing a single value you have to include a comma, even though there is only one value –

```
tup1 = (50,);
```

Like string indices, tuple indices start at 0, and they can be sliced, concatenated, and so on.

• Accessing Values in Tuples:

To access values in tuple, use the square brackets for slicing along with the index or indices to obtain value available at that index. For example –

```
tup1 = ('physics', 'chemistry', 1997, 2000);
tup2 = (1, 2, 3, 4, 5, 6, 7 ); print "tup1[0]:
", tup1[0]

print "tup2[1:5]: ", tup2[1:5]
```

When the code is executed, it produces the following result –

```
tup1[0]: physics
tup2[1:5]: [2, 3, 4, 5]
```

UPDATING TUPLES:

Tuples are immutable which means you cannot update or change the values of tuple elements. We are able to take portions of existing tuples to create new tuples as the following example demonstrates –

```
tup1 = (12, 34.56);
tup2 = ('abc', 'xyz');
tup3 = tup1 + tup2;
print tup3
```

When the above code is executed, it produces the following result –

```
(12, 34.56, 'abc', 'xyz')
```

DELETE TUPLE ELEMENTS

Removing individual tuple elements is not possible. There is, of course, nothing wrong with putting together another tuple with the undesired elements discarded.

To explicitly remove an entire tuple, just use the **del** statement. For example:

```
tup = ('physics', 'chemistry', 1997, 2000);
print tup
del tup
print "After deleting
tup : "
print tup
```

BASIC TUPLES OPERATIONS:

SN	Function with Description
1	<code>cmp(tuple1, tuple2)</code> :Compares elements of both tuples.
2	<code>len(tuple)</code> :Gives the total length of the tuple.
3	<code>max(tuple)</code> :Returns item from the tuple with max value.
4	<code>min(tuple)</code> :Returns item from the tuple with min value.
5	<code>tuple(seq)</code> :Converts a list into tuple.

Python Expression	Results	Description
<code>len((1, 2, 3))</code>	3	Length
<code>(1, 2, 3) + (4, 5, 6)</code>	<code>(1, 2, 3, 4, 5, 6)</code>	Concatenation
<code>('Hi!') * 4</code>	<code>('Hi!', 'Hi!', 'Hi!', 'Hi!')</code>	Repetition
<code>3 in (1, 2, 3)</code>	True	Membership
<code>for x in (1, 2, 3): print x,</code>	1 2 3	Iteration

Built-in Tuple Functions

DICTIONARY

Each key is separated from its value by a colon (:), the items are separated by commas, and the whole thing is enclosed in curly braces. An empty dictionary without any items is written with just two curly braces, like this: {}.

Keys are unique within a dictionary while values may not be. The values of a dictionary can be of any type, but the keys must be of an immutable data type such as strings, numbers, or tuples.

ACCESSING VALUES IN DICTIONARY:

To access dictionary elements, you can use the familiar square brackets along with the key to obtain its value. Following is a simple example –

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}

print "dict['Name']: ", dict['Name']
print "dict['Age']: ", dict['Age']
```

Result –

```
dict['Name']: Zara dict['Age']:
7
```

UPDATING DICTIONARY

We can update a dictionary by adding a new entry or a key-value pair, modifying an existing entry, or deleting an existing entry as shown below in the simple example –

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}

dict['Age'] = 8; # update existing entry
dict['School'] = "DPS School"; # Add new entry
print "dict['Age']: ", dict['Age'],
print "dict['School']: ", dict['School']
```

Result –

```
dict['Age']: 8  
dict['School']: DPS School
```

DELETE DICTIONARY ELEMENTS

We can either remove individual dictionary elements or clear the entire contents of a dictionary. You can also delete entire dictionary in a single operation.

To explicitly remove an entire dictionary, just use the **del** statement. Following is a simple example –

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}  
  
del dict['Name']; # remove entry with key  
  
'Name' dict.clear(); # remove all entries in dict  
  
del dict ;      # delete entire dictionary  
  
print "dict['Age']: ", dict['Age'] print  
"dict['School']: ", dict['School']
```


Built-in Dictionary Functions & Methods –

Python includes the following dictionary functions –

SN	Function with Description
1	cmp(dict1, dict2) Compares elements of both dict.
2	len(dict) Gives the total length of the dictionary. This would be equal to the number of items in dictionary.
3	str(dict) Produces a printable string representation of a dictionary
4	type(variable) Returns the type of the passed variable. If passed variable is dictionary, then it would return a dictionary type.

Python includes following dictionary methods –

SN	Methods with Description
1	dict.clear() : Removes all elements of dictionary <i>dict</i>

2	<u>dict. Copy()</u> :Returns a shallow copy of dictionary <i>dict</i>
3	<u>dict.fromkeys()</u> :Create a new dictionary with keys from seq and values <i>set</i> to <i>value</i> .
4	<u>dict.get(key, default=None)</u> :For <i>key</i> key, returns value or default if key not in dictionary
5	<u>dict.has_key(key)</u> :Returns <i>true</i> if key in dictionary <i>dict</i> , <i>false</i> otherwise
6	<u>dict.items()</u> :Returns a list of <i>dict</i> 's (key, value) tuple pairs
7	<u>dict.keys()</u> :Returns list of dictionary <i>dict</i> 's keys
8	<u>dict.setdefault(key, default=None)</u> :Similar to <i>get()</i> , but will set <i>dict[key]=default</i> if <i>key</i> is not already in <i>dict</i>
9	<u>dict.update(dict2)</u> :Adds dictionary <i>dict2</i> 's key-values pairs to <i>dict</i>
10	<u>dict.values()</u> :Returns list of dictionary <i>dict</i> 's values

A function is a block of organized, reusable code that is used to perform a single, related action. Functions provide better modularity for your application and a high degree of code reusing. Python gives you many built-in functions like *print()*, etc. but you can also create your own functions. These functions are called user-defined functions.

DEFINING A FUNCTION

Simple rules to define a function in Python.

- Function blocks begin with the keyword `def` followed by the function name and parentheses `( )`.
- Any input parameters or arguments should be placed within these parentheses. You can also define parameters inside these parentheses.
- The first statement of a function can be an optional statement - the documentation string of the function or *docstring*.
- The code block within every function starts with a colon `(:)` and is indented.
- The statement `return [expression]` exits a function, optionally passing back an expression to the caller. A return statement with no arguments is the same as `return None`.

```
def functionname( parameters ):

    "function_docstring"
    function_suite    return
    [expression]
```

CALLING A FUNCTION

Defining a function only gives it a name, specifies the parameters that are to be included in the function and structures the blocks of code. Once the basic structure of a function is finalized, you can execute it by calling it from another function or directly from the Python prompt. Following is the example to call `printme()` function –

```
# Function definition is here

def printme( str ):

    "This prints a passed string into this function"
    print str    return; printme("I'm first call to user
defined function!") printme("Again second call to
the same function")
```

When the above code is executed, it produces the following result –

```
I'm first call to user defined function!  
Again second call to the same function
```

Function Arguments

You can call a function by using the following types of formal arguments:

- Required arguments
- Keyword arguments
- Default arguments
- Variable-length arguments

Scope of Variables

All variables in a program may not be accessible at all locations in that program. This depends on where you have declared a variable.

The scope of a variable determines the portion of the program where you can access a particular identifier. There are two basic scopes of variables in Python –

Global variables

Local variables

Global vs. Local variables

Variables that are defined inside a function body have a local scope, and those defined outside have a global scope.

This means that local variables can be accessed only inside the function in which they are declared, whereas global variables can be accessed throughout the program body by all functions. When you call a function, the variables declared inside it are brought into scope.

Following is a simple example –

```
total = 0; # This is global variable.  
  
# Function definition is here def  
sum( arg1, arg2 ):
```

```
# Add both the parameters and return them."

total = arg1 + arg2; # Here total is local variable.

print "Inside the function local total : ", total
return total; sum( 10, 20 ); print "Outside the
function global total : ", total
```

Result –

```
Inside the function local total : 30
```

```
Outside the function global total : 0
```

A module allows you to logically organize your Python code. Grouping related code into a module makes the code easier to understand and use. A module is a Python object with arbitrarily named attributes that you can bind and reference. Simply, a module is a file consisting of Python code. A module can define functions, classes and variables. A module can also include runnable code.

Example:

The Python code for a module named *aname* normally resides in a file named *aname.py*. Here's an example of a simple module, support.py

```
def print_func( par ):

    print "Hello : ", par

    return
```

THE IMPORT STATEMENT

The *import* has the following syntax:

```
import module1[, module2[,... moduleN]
```

When the interpreter encounters an import statement, it imports the module if the module is present in the search path. A search path is a list of directories that the interpreter searches before importing a module. For example, to import the module `support.py`, you need to put the following command at the top of the script –

A module is loaded only once, regardless of the number of times it is imported. This prevents the module execution from happening over and over again if multiple imports occur.

PACKAGES IN PYTHON

A package is a hierarchical file directory structure that defines a single Python application environment that consists of modules and sub packages and sub-sub packages.

Consider a file *Pots.py* available in *Phone* directory. This file has following line of source code –

```
def Pots():  
    print "I'm Pots Phone"
```

Similar way, we have another two files having different functions with the same name as above –

- Phone/Isdn.py file having function Isdn()
- Phone/G3.py file having function G3()

Now, create one more file `__init__.py` in Phone directory –

- Phone/`__init__.py`

To make all of your functions available when you've imported Phone, to put explicit import statements in `__init__.py` as follows –

```

from Pots import Pots from

Isdn import Isdn from G3

import G3

```

After you add these lines to `__init__.py`, you have all of these classes available when you import the Phone package.

```

# Now import your Phone Package.

import Phone Phone.Pots()

Phone.Isdn()

Phone.G3()

```

RESULT:

```

I'm Pots Phone I'm

3G Phone

I'm ISDN Phone

```

In the above example, we have taken example of a single functions in each file, but you can keep multiple functions in your files. You can also define different Python classes in those files and then you can create your packages out of those classes.

This chapter covers all the basic I/O functions available in Python.

Printing to the Screen

The simplest way to produce output is using the *print* statement where you can pass zero or more expressions separated by commas. This function converts the expressions you pass into a string and writes the result to standard output as follows –

```

print "Python is really a great language,", "isn't it?"

```

Result:

Python is really a great language, isn't it?

Reading Keyboard Input

Python provides two built-in functions to read a line of text from standard input, which by default comes from the keyboard. These functions are –

- `raw_input`
- `input`

The `raw_input` Function

The `raw_input([prompt])` function reads one line from standard input and returns it as a string (removing the trailing newline).

```
str = raw_input("Enter your input: ");  
print "Received input is : ", str
```

This prompts you to enter any string and it would display same string on the screen. When I typed "Hello Python!", its output is like this –

```
Enter your input: Hello Python  
Received input is : Hello Python
```

The `input` Function

The `input([prompt])` function is equivalent to `raw_input`, except that it assumes the input is a valid Python expression and returns the evaluated result to you.

```
str = input("Enter your input: "); print  
"Received input is : ", str
```

This would produce the following result against the entered input –

```
Enter your input: [x*5 for x in range(2,10,2)]  
Received input is : [10, 20, 30, 40]
```


Opening and Closing Files

Until now, you have been reading and writing to the standard input and output. Now, we will see how to use actual data files.

Python provides basic functions and methods necessary to manipulate files by default. You can do most of the file manipulation using a **file** object.

The *open* Function

Before you can read or write a file, you have to open it using Python's built-in *open()* function. This function creates a **file** object, which would be utilized to call other support methods associated with it. Syntax

```
file object = open(file_name [, access_mode][, buffering])
```

Here are parameter details:

- **file_name:** The file_name argument is a string value that contains the name of the file that you want to access.
- **access_mode:** The access_mode determines the mode in which the file has to be opened, i.e., read, write, append, etc. A complete list of possible values is given below in the table. This is optional parameter and the default file access mode is read (r).
- **buffering:** If the buffering value is set to 0, no buffering takes place. If the buffering value is 1, line buffering is performed while accessing a file. If you specify the buffering value as an integer greater than 1, then buffering action is performed with the indicated buffer size. If negative, the buffer size is the system default(default behavior).

Here is a list of the different modes of opening a file –

Modes	Description
r	Opens a file for reading only. The file pointer is placed at the beginning of the file. This is the default mode.

rb	Opens a file for reading only in binary format. The file pointer is placed at the beginning of the file. This is the default mode.
r+	Opens a file for both reading and writing. The file pointer placed at the beginning of the file.
rb+	Opens a file for both reading and writing in binary format. The file pointer placed at the beginning of the file.
w	Opens a file for writing only. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.
wb	Opens a file for writing only in binary format. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.
w+	Opens a file for both writing and reading. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.
wb+	Opens a file for both writing and reading in binary format. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.
a	Opens a file for appending. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.

ab	Opens a file for appending in binary format. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.
a+	Opens a file for both appending and reading. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing.
ab+	Opens a file for both appending and reading in binary format. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing.

The *file* Object Attributes

Once a file is opened and you have one *file* object, you can get various information related to that file.

Here is a list of all attributes related to file object:

Attribute	Description
file.closed	Returns true if file is closed, false otherwise.
file.mode	Returns access mode with which file was opened.
file.name	Returns name of the file.

file.softspace	Returns false if space explicitly required with print, true otherwise.
----------------	--

Example

```
# Open a file fo = open("foo.txt",
"wb") print "Name of the file: ",
fo.name print "Closed or not : ",
fo.closed print "Opening mode : ",
fo.mode print "Softspace flag : ",
fo.softspace
```

This produces the following result –

```
Name of the file: foo.txt
Closed or not : False
Opening mode : wb
Softspace flag : 0
```

The close() Method

The close() method of a *file* object flushes any unwritten information and closes the file object, after which no more writing can be done. Python automatically closes a file when the reference object of a file is reassigned to another file. It is a good practice to use the close() method to close a file

Syntax

```
fileObject.close();
```

Example

```
# Open a file fo =
open("foo.txt", "wb")
```

```
print "Name of the file: ", fo.name  
  
# Close opened file  
fo.close()
```

Result –

```
Name of the file: foo.txt
```

Reading and Writing Files

The *file* object provides a set of access methods to make our lives easier. We would see how to use *read()* and *write()* methods to read and write files.

The write() Method

The *write()* method writes any string to an open file. It is important to note that Python strings can have binary data and not just text. The *write()* method does not add a newline character ('\n') to the end of the string **Syntax**

```
fileObject.write(string);
```

Here, passed parameter is the content to be written into the opened file. **Example**

```
# Open a file fo = open("foo.txt", "wb") fo.write( "Python is  
a great language.\nYeah its great!!\n");  
  
# Close opened file  
fo.close()
```

The above method would create *foo.txt* file and would write given content in that file and finally it would close that file. If you would open this file, it would have following content.

```
Python is a great language.  
Yeah its great!!
```

The read() Method

The *read()* method reads a string from an open file. It is important to note that Python strings can have binary data. apart from text data. Syntax

```
fileObject.read([count]);
```

Here, passed parameter is the number of bytes to be read from the opened file. This method starts reading from the beginning of the file and if *count* is missing, then it tries to read as much as possible, maybe until the end of file.

Example

Let's take a file *foo.txt*, which we created above.

```
# Open a file fo =  
open("foo.txt", "r+") str =  
fo.read(10); print "Read  
String is : ", str # Close  
opened file fo.close()
```

This produces the following result –

```
Read String is : Python is
```

File Positions

The *tell()* method tells you the current position within the file; in other words, the next read or write will occur at that many bytes from the beginning of the file.

32

The *seek(offset[, from])* method changes the current file position. The *offset* argument indicates the number of bytes to be moved. The *from* argument specifies the reference position from where the bytes are to be moved.

If *from* is set to 0, it means use the beginning of the file as the reference position and 1 means use the current position as the reference position and if it is set to 2 then the end of the file would be taken as the reference position.

Example

Let us take a file *foo.txt*, which we created above.

```
# Open a file fo =
open("foo.txt", "r+") str =
fo.read(10); print "Read
String is : ", str

# Check current position position =
fo.tell(); print "Current file position : ",
position

# Reposition pointer at the beginning once
again position = fo.seek(0, 0); str = fo.read(10);
print "Again read String is : ", str

# Close opened file
fo.close()
```

This produces the following result –

```
Read String is : Python is
Current file position : 10
Again read String is : Python is
```

Renaming and Deleting Files

Python **os** module provides methods that help you perform file-processing operations, such as renaming and deleting files.

To use this module you need to import it first and then you can call any related functions.

The `rename()` Method

The *rename()* method takes two arguments, the current filename and the new filename. Syntax

```
os.rename(current_file_name, new_file_name)
```

Example

Following is the example to rename an existing file *test1.txt*:

```
import os

# Rename a file from test1.txt to test2.txt os.rename( "test1.txt", "test2.txt" )
```

The `remove()` Method

You can use the *remove()* method to delete files by supplying the name of the file to be deleted as the argument. Syntax

```
os.remove(file_name)
```

Example

Following is the example to delete an existing file *test2.txt* –

```
#!/usr/bin/python import os

# Delete file test2.txt os.remove("test2.txt")
```

Directories in Python

All files are contained within various directories, and Python has no problem handling these too. The **os** module has several methods that help you create, remove, and change directories.

The `mkdir()` Method

You can use the `mkdir()` method of the `os` module to create directories in the current directory. You need to supply an argument to this method which contains the name of the directory to be created. Syntax

```
os.mkdir("newdir")
```

Example

Following is the example to create a directory `test` in the current directory –

```
#!/usr/bin/python import os

# Create a directory "test" os.mkdir("test")
```

The `chdir()` Method

You can use the `chdir()` method to change the current directory. The `chdir()` method takes an argument, which is the name of the directory that you want to make the current directory.

Syntax

```
os.chdir("newdir")
```

Example

Following is the example to go into `"/home/newdir"` directory –

```
#!/usr/bin/python

import os

# Changing a directory to "/home/newdir" os.chdir("/home/newdir")
```

The `getcwd()` Method

The `getcwd()` method displays the current working directory. Syntax

```
os.getcwd()
```

Example

Following is the example to give current directory –

```
import os

# This would give location of the current directory os.getcwd()
```

The rmdir() Method

The *rmdir()* method deletes the directory, which is passed as an argument in the method.

Before removing a directory, all the contents in it should be removed.

Syntax:

```
os.rmdir('dirname')
```

Example

Following is the example to remove `"/tmp/test"` directory. It is required to give fully qualified name of the directory, otherwise it would search for that directory in the current directory.

```
import os

# This would remove "/tmp/test" directory.
os.rmdir( "/tmp/test" )
```

File & Directory Related Methods

There are three important sources, which provide a wide range of utility methods to handle and manipulate files & directories on Windows and Unix operating systems. They are as follows –

- File Object Methods: The *file* object provides functions to manipulate files.
- OS Object Methods: This provides methods to process files as well as directories.

Python provides two very important features to handle any unexpected error in your Python programs and to add debugging capabilities in them –

- **Exception Handling**: This would be covered in this tutorial. Here is a list standard Exceptions available in Python: Standard Exceptions.

- **Assertions:** This would be covered in [Assertions in Python](#)

List of Standard Exceptions –

EXCEPTION NAME	DESCRIPTION
Exception	Base class for all exceptions
StopIteration	Raised when the next() method of an iterator does not point to any object.
SystemExit	Raised by the sys.exit() function.
StandardError	Base class for all built-in exceptions except StopIteration and SystemExit.
ArithmeticError	Base class for all errors that occur for numeric calculation.
OverflowError	Raised when a calculation exceeds maximum limit for a numeric type.
FloatingPointError	Raised when a floating point calculation fails.
ZeroDivisionError	Raised when division or modulo by zero takes place for all numeric types.

AssertionError	Raised in case of failure of the Assert statement.
AttributeError	Raised in case of failure of attribute reference or assignment.
EOFError	Raised when there is no input from either the raw_input() or input() function and the end of file is reached.
ImportError	Raised when an import statement fails.
KeyboardInterrupt	Raised when the user interrupts program execution, usually by pressing Ctrl+c.
LookupError	Base class for all lookup errors.

IndexError	Raised when an index is not found in a sequence.
KeyError	Raised when the specified key is not found in the dictionary.
NameError	Raised when an identifier is not found in the local or global namespace.
UnboundLocalError	Raised when trying to access a local variable in a function or method but no value has been assigned to it.
EnvironmentError	Base class for all exceptions that occur outside the Python environment.

IOError IOError	Raised when an input/ output operation fails, such as the print statement or the open() function when trying to open a file that does not exist. Raised for operating system-related errors.
SyntaxError IndentationError	Raised when there is an error in Python syntax. Raised when indentation is not specified properly.
SystemError	Raised when the interpreter finds an internal problem, but when this error is encountered the Python interpreter does not exit.
SystemExit	Raised when Python interpreter is quit by using the sys.exit() function. If not handled in the code, causes the interpreter to exit.
TypeError	Raised when an operation or function is attempted that is invalid for the specified data type.
ValueError	Raised when the built-in function for a data type has the valid type of arguments, but the arguments have invalid values specified.
RuntimeError	Raised when a generated error does not fall into any category.
NotImplementedError	Raised when an abstract method that needs to be implemented in an inherited class is not actually implemented.

What is Exception?

An exception is an event, which occurs during the execution of a program that disrupts the normal flow of the program's instructions. In general, when a Python script encounters a

situation that it cannot cope with, it raises an exception. An exception is a Python object that represents an error.

When a Python script raises an exception, it must either handle the exception immediately otherwise it terminates and quits.

Handling an exception

If you have some *suspicious* code that may raise an exception, you can defend your program by placing the suspicious code in a **try:** block. After the try: block, include an **except:** statement, followed by a block of code which handles the problem as elegantly as possible.

The Python standard for database interfaces is the Python DB-API. Most Python database interfaces adhere to this standard.

You can choose the right database for your application. Python Database API supports a wide range of database servers such as –

- GadFly
- mSQL
- MySQL
- PostgreSQL
- Microsoft SQL Server 2000
- Informix
- Interbase
- Oracle
- Sybase

The DB API provides a minimal standard for working with databases using Python structures and syntax wherever possible. This API includes the following:

- Importing the API module.
- Acquiring a connection with the database.
- Issuing SQL statements and stored procedures.
- Closing the connection

9.2 SOURCE CODE

HOME PAGE.PY

```

from tkinter import messagebox from tkinter import

* from tkinter import simpledialog import tkinter

import matplotlib.pyplot as plt import numpy as np

from tkinter import ttk from tkinter import

filedialog import pandas as pd from

sklearn.model_selection import train_test_split

import os import cv2 from sklearn.preprocessing

import StandardScaler from sklearn.metrics import

accuracy_score from sklearn import svm from

sklearn.metrics import accuracy_score from

sklearn.metrics import f1_score from

sklearn.metrics import precision_score from

sklearn.metrics import recall_score import

matplotlib.pyplot as plt import seaborn as sns

import os from sklearn.metrics import

```

```

confusion_matrix main = Tk() main.title("Children
ADHD Disease Detection using Pose Extimation
Technique") main.geometry("1300x1200") global
filename global X, Y global X_train, X_test,
y_train, y_test, scaler, svm_cls global dataset
proto_File = "Models/pose_deploy_linevec.prototxt"
weights_File = "Models/pose_iter_440000.caffemodel" n_Points
= 18
POSE_PAIRS = [
[1,0],[1,2],[1,5],[2,3],[3,4],[5,6],[6,7],[1,8],[8,9],[9,10],[1,11],[11,12],[12,13],[0,14],[0,15],[1
4,16],[15,17]] in_Width
= 368 in_Height = 368
threshold = 0.1
POSE_NAMES = ["Head", "Neck", "RShoulder", "RElbow", "RWrist", "LShoulder",
"LElbow", "LWrist", "RHip", "RKnee",
"RAnkle", "LHip", "LKnee", "LAnkle", "Chest", "Background"] net
= cv2.dnn.readNetFromCaffe(proto_File, weights_File)
net.setPreferableBackend(cv2.dnn.DNN_TARGET_CPU)
def uploadDataset():
    global filename, dataset    filename =
    filedialog.askopenfilename(initialdir="ADHDDataset")
    text.delete('1.0', END)    text.insert(END,filename+" loaded\n\n")

```



```

dataset = pd.read_csv(filename)

text.insert(END,str(dataset.head()))

def processDataset():

global dataset, X, Y, scaler

text.delete('1.0', END)

data = dataset.values

X = data[:,0:data.shape[1]-1]  Y = data[:,data.shape[1]-
1]  indices = np.arange(X.shape[0]) #shuffling dataset

values  np.random.shuffle(indices)

X = X[indices]  Y = Y[indices]  scaler = StandardScaler()  X =
scaler.fit_transform(X)  text.insert(END,"Dataset Processing, Shuffling &
Normalization Completed\n\n")  text.insert(END,"Normalized Dataset Values =
"+str(X)) def splitDataset():

text.delete('1.0', END)      global X, Y,

X_train, X_test, y_train, y_test

X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2)

text.insert(END,"Total records found in Dataset : "+str(X.shape[0])+"\n")

text.insert(END,"Total features found in each record : "+str(X.shape[1])+"\n")

text.insert(END,"Dataset Train & Test Split\n")  text.insert(END,"80% dataset size used to
train algorithms : "+str(X_train.shape[0])+"\n")  text.insert(END,"20% dataset size used to

```

```

test algorithms : "+str(X_test.shape[0])+"\n") def calculateMetrics(algorithm, predict,
y_test):

    label = ['Normal', 'ADHD Disease']          a =
accuracy_score(y_test,predict)*100              p =
precision_score(y_test, predict,average='macro') * 100    r =
recall_score(y_test, predict,average='macro') * 100      f =
f1_score(y_test,    predict,average='macro')    *    100

text.insert(END,algorithm+" Accuracy   :   "+str(a)+"\n")

text.insert(END,algorithm+" Precision  :   "+str(p)+"\n")

text.insert(END,algorithm+" Recall     :   "+str(r)+"\n")

text.insert(END,algorithm+" FScore     :   "+str(f)+"\n")

conf_matrix    =    confusion_matrix(y_test,    predict)

plt.figure(figsize =(6, 3))

    ax = sns.heatmap(conf_matrix, xticklabels = label, yticklabels = label, annot = True,
cmap="viridis" ,fmt ="g");    ax.set_ylim([0,len(label)])    plt.title(algorithm+" Confusion
matrix")    plt.xticks(rotation=90)    plt.ylabel('True class')

    plt.xlabel('Predicted class')    plt.show()    def trainSVM():    global
svm_cls    text.delete('1.0', END)    global X_train, X_test, y_train, y_test

svm_cls = svm.SVC(kernel="rbf", C = 12, probability=True, gamma="auto")

svm_cls.fit(X_train, y_train)    predict = svm_cls.predict(X_test)

calculateMetrics("SVM", predict, y_test) def predictADHD(testData):

```

```

values = []    testData =

np.asarray(testData)

values.append(testData)    testData =

np.asarray(values)    testData =

scaler.transform(testData)    predict =

svm_cls.predict(testData)    return

int(predict[0]) def

detectDisease(frame):

    global net
    frame_Width = frame.shape[1]    frame_Height =

frame.shape[0]    img = np.zeros((frame_Height,frame_Width,3),

dtype=np.uint8)

    inp_Blob = cv2.dnn.blobFromImage(frame, 1.0 / 255, (in_Width, in_Height), (0, 0, 0),

swapRB=False, crop=False)    net.setInput(inp_Blob)    output = net.forward()    H =

output.shape[2]    W = output.shape[3]    points = []    testData = []    for i in range(n_Points):

        probMap = output[0, i, :, :]    minVal, prob, minLoc,

point = cv2.minMaxLoc(probMap)    x = (frame_Width *

point[0]) / W    y = (frame_Height * point[1]) / H    if

prob > threshold :

        cv2.circle(frame,    (int(x),    int(y)),    8,    (0,    255,    255),

thickness=-1, lineType=cv2.FILLED)

        cv2.putText(frame,"{}".format(i), (int(x), int(y)), cv2.FONT_HERSHEY_SIMPLEX,

```

```

1, (0, 0, 255), 2, lineType=cv2.LINE_AA)

    points.append((int(x), int(y)))

testData.append(x)        testData.append(y)

    else :

        points.append(None)

testData.append(0)

testData.append(0)    predict =

predictADHD(testData)        for

pair in POSE_PAIRS:

    partA = pair[0]    partB = pair[1]    print(str(pair[0])+"

"+str(pair[1])+" "+str(partA)+" "+str(partB))    if points[partA]

and points[partB]:

    cv2.line(img, points[partA], points[partB], (0, 255, 255), 3, lineType=cv2.LINE_AA)

    cv2.line(frame,    points[partA], points[partB], (0,    255,    255), 3,

lineType=cv2.LINE_AA)

    cv2.circle(frame, points[partA], 8, (0, 0, 255), thickness=-1, lineType=cv2.FILLED)

cv2.circle(frame, points[partB], 8, (0, 0, 255), thickness=-1, lineType=cv2.FILLED)    return

frame, img, predict

def imageDetect():

    text.delete('1.0', END)    global scaler, svm_cls    filename

= filedialog.askopenfilename(initialdir="images")    frame =

```

```

cv2.imread(filename)    frame, img, predict =

detectDisease(frame)    frame = cv2.resize(frame, (400,

400))    img = cv2.resize(img, (400, 400))    label =

['Normal', 'ADHD Disease']    print(predict)

    cv2.putText(frame, 'Predicted    As    :    '+label[predict],    (10,    25),

cv2.FONT_HERSHEY_SIMPLEX,0.7, (0, 0, 255), 2)    cv2.imshow("Pose Estimated

Image", frame)    cv2.imshow("Pose Image", img)    cv2.waitKey(0) def videoDetect():

    text.delete('1.0', END)    global scaler, svm_cls

filename = filedialog.askopenfilename(initialdir="videos")

normal_count = 0    abnormal_count = 0    video =

cv2.VideoCapture(filename)

    count        =        0

while(True):

    ret, frame = video.read()

if ret == True:

        filename = "temp.png"        frame, img,

predict = detectDisease(frame)        if predict

== 0:        normal_count += 1

        else:

```

```

        abnormal_count + 1

cv2.imshow("Estimated Pose", frame)        if

cv2.waitKey(5) & 0xFF == ord('q'):

        break

count = count + 1        if

count > 20:        break

else:        break

video.release()

cv2.destroyAllWindows() if

normal_count >=

abnormal_count:

        text.insert(END,"Pose in Video Predicted as : NORMAL\n")

else:

        text.insert(END,"Pose in Video Predicted as : ADHD Disease\n") font

= ('times', 15, 'bold')

title = Label(main, text='Children ADHD Disease Detection using Pose Estimation Technique')

title.config(bg='darkviolet', fg='gold')    title.config(font=font)        title.config(height=3,

width=120)    title.place(x=0,y=5) font1 = ('times', 13, 'bold') ff = ('times', 12, 'bold')

uploadButton = Button(main, text="Upload ADHD Pose Dataset", command=uploadDataset)

uploadButton.place(x=20,y=100) uploadButton.config(font=ff) processButton =

```

```

Button(main, text="Preprocess Dataset", command=processDataset)

processButton.place(x=20,y=150) processButton.config(font=ff) splitButton = Button(main,
text="Split Dataset Train & Test", command=splitDataset) splitButton.place(x=20,y=200)
splitButton.config(font=ff) svmButton = Button(main, text="Train SVM
Algorithm", command=trainSVM) svmButton.place(x=20,y=250)
svmButton.config(font=ff)

imageDetectionButton = Button(main, text="Disease Detection from Test Image",
command=imageDetect) imageDetectionButton.place(x=20,y=300)
imageDetectionButton.config(font=ff)

videoDetectionButton = Button(main, text="Disease Detection from Video",
command=videoDetect) videoDetectionButton.place(x=20,y=350)
videoDetectionButton.config(font=ff) font1 = ('times', 12, 'bold')
text=Text(main,height=30,width=110) scroll=Scrollbar(text)
text.configure(yscrollcommand=scroll.set) text.place(x=360,y=100) text.config(font=font1)
main.config(bg='forestgreen')

main.mainloop()

```

CHAPTER-10

RESULTS/DISCUSSION

10.1 SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub-assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

TYPES OF TESTS

UNIT TESTING

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application. It is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

INTEGRATION TESTING

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfactory, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

FUNCTIONAL TEST

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

- Valid Input : identified classes of valid input must be accepted.
- Invalid Input : identified classes of invalid input must be rejected.
- Functions : identified functions must be exercised.
- Output : identified classes of application outputs must be exercised.
- Systems/Procedures : interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

SYSTEM TEST

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration-oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

WHITE BOX TESTING

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

BLACK BOX TESTING

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

UNIT TESTING

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

TEST STRATEGY AND APPROACH

Field testing will be performed manually and functional tests will be written in detail.

Test objectives

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Features to be tested

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects. The task of the integration test is to check that components or software applications,

e.g. components in a software system or – one step up – software applications at the company level – interact without error.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

ACCEPTANCE TESTING

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

10.1.1 TEST CASES

Test case1:

Test case for Login form:

FUNCTION:	LOGIN
EXPECTED RESULTS:	Should Validate the user and check his existence in database
ACTUAL RESULTS:	Validate the user and checking the user against the database
LOW PRIORITY	No
HIGH PRIORITY	Yes

Test case2:

Test case for User Registration form:

FUNCTION	USER REGISTRATION
----------	-------------------

EXPECTED RESULTS:	Should check if all the fields are filled by the user and saving the user to database.
ACTUAL RESULTS:	Checking whether all the fields are filled by user or not through validations and saving user.
LOW PRIORITY	No
HIGH PRIORITY	Yes

Test case3:

Test case for Change Password:

When the old password does not match with the new password , then this results in displaying an error message as “ OLD PASSWORD DOES NOT MATCH WITH THE NEW PASSWORD”.

Test case 4:

Test case for Forget Password:

When a user forgets his password he is asked to enter Login name, ZIP code, Mobile number. If these are matched with the already stored ones then user will get his Original password.

Module	Functionality	Test Case	Expected Results	Actual Results	Result	Priority

User	Login Usecase	1. Navigate To Www.Sample.Co m 2. Click On Submit Button Without Entering Username and Password	A Validation Should Be As Below “Please Enter Valid Username & Password”	A Validation Has Been Populated As Expected	Pass	High
		1. aNavigate To Www.Sample.Co m 2. Click On Submit Button With Out Filling Password And With Valid Username	A Validation Should Be As Below “Please Enter Valid Password Or Password Field Can Not Be Empty “	A Validation Is Shown As Expected	Pass	High
		1. NNavigate To Www.Sample.Co m 2. Enter Both Username And Password Wrong And Hit Enter	A Validation Shown As Below “The Username Entered Is Wrong”	A Validation Is Shown As Expected	Pass	High

		1. Navigate To Www.Sample.Co m	Validate Username And Password In DataBase And Once If They Correct Then Show The Main Page	Main Page/ Home Page Has Been Displayed	Pass	High
		2. Enter Validate Username And Password And Click On Submit				

10.2 SCREENSHOTS

FIG 1: HOME PAGE

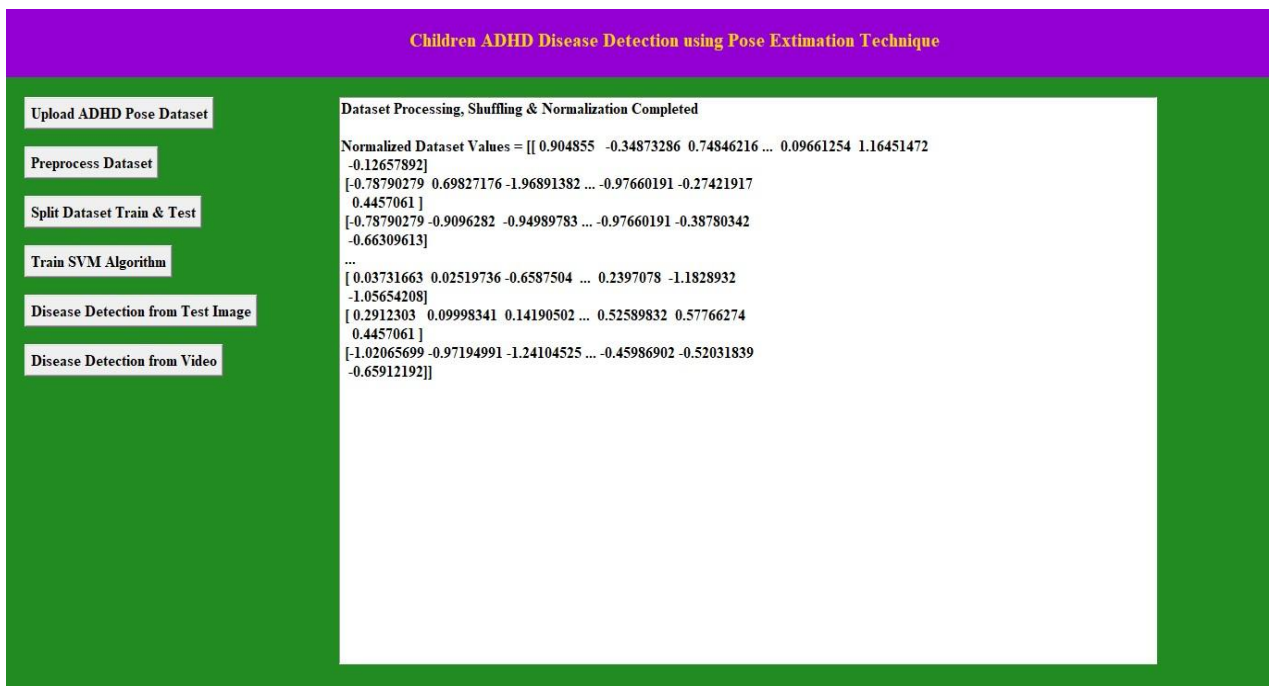


FIG
2: POSE DATASET PAGE

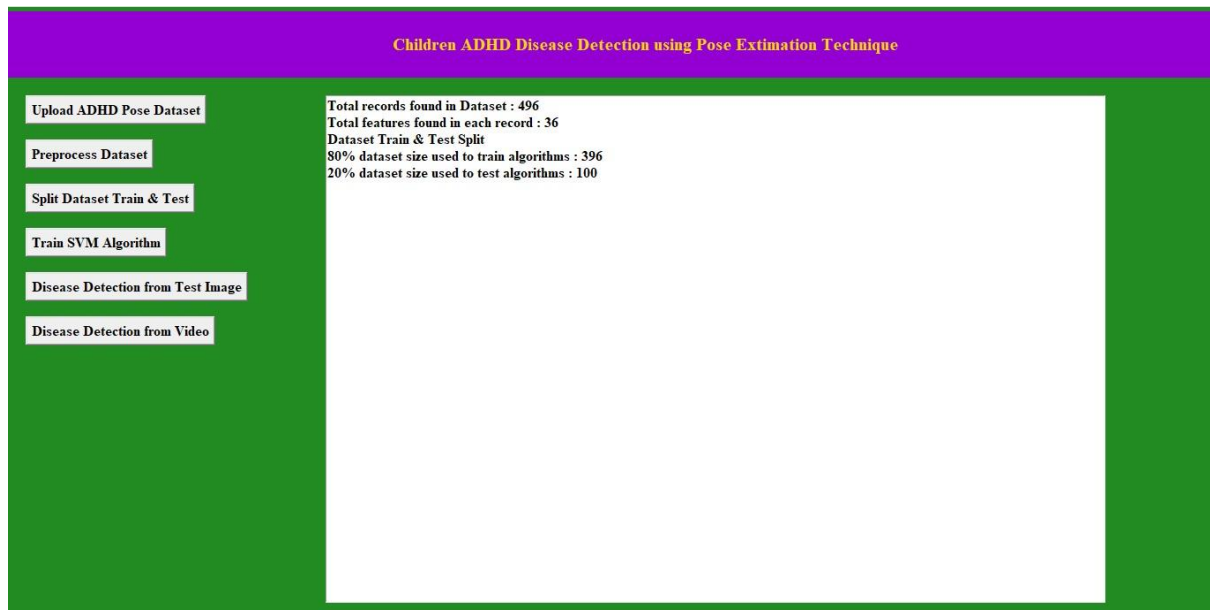
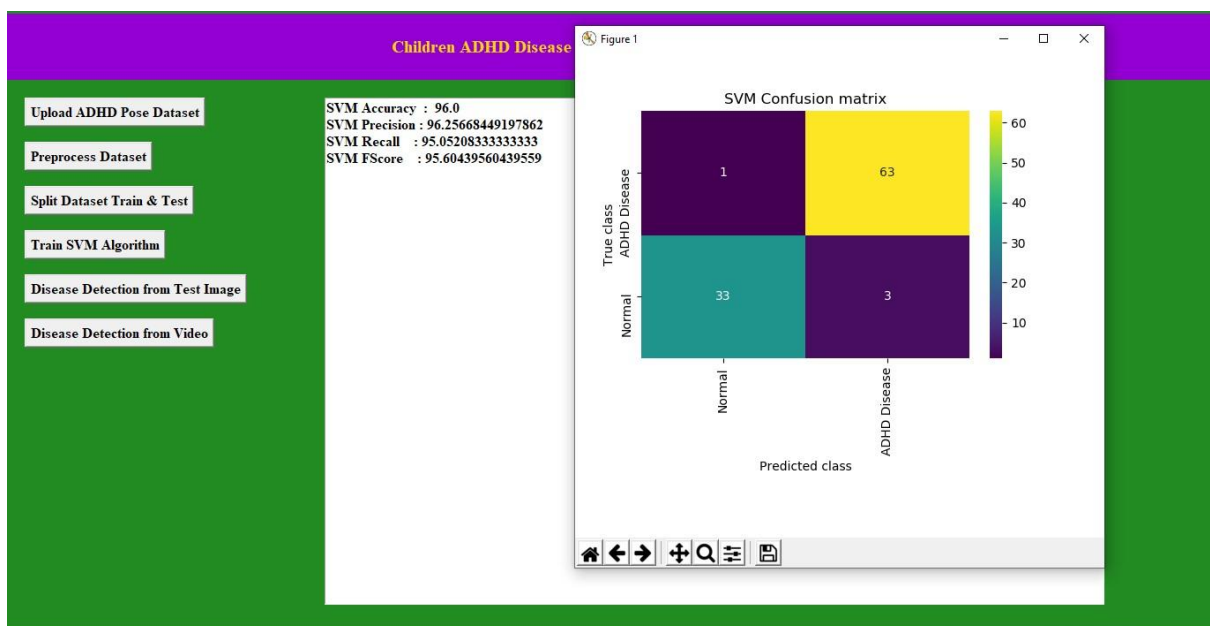


FIG 3: POSE PREDICTION PAGE



FIG

4: IMAGE UPLOADING PAGE

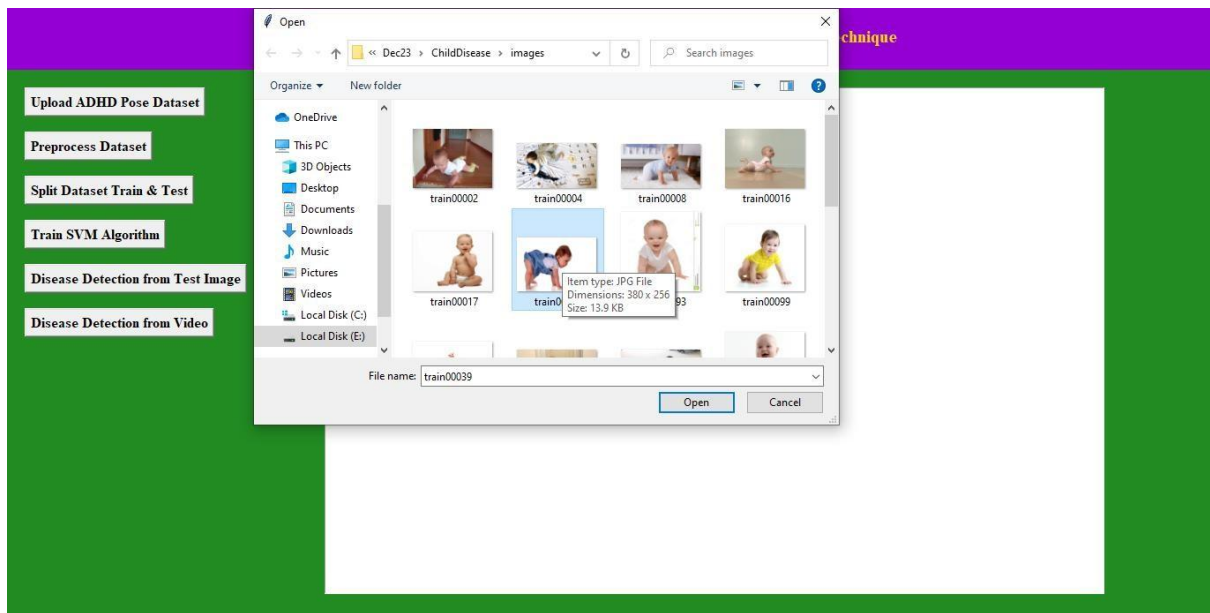


FIG 5: POSE ESTIMATION PAGE

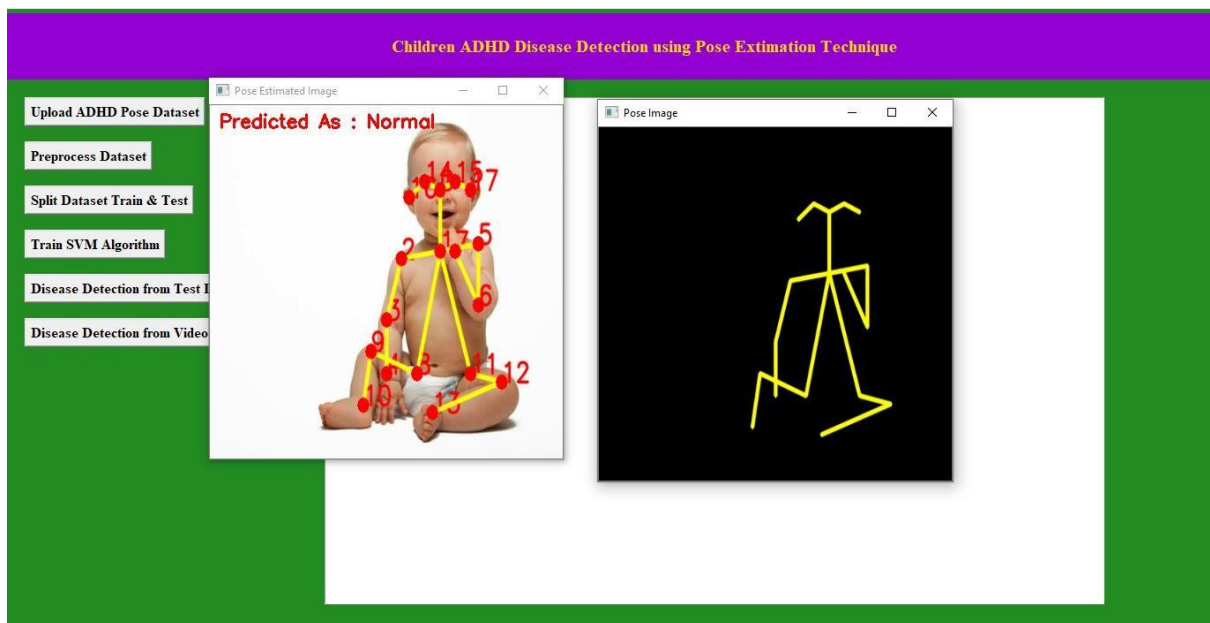


FIG
6: POSE ADHD DISEASE DETECTION PAGE

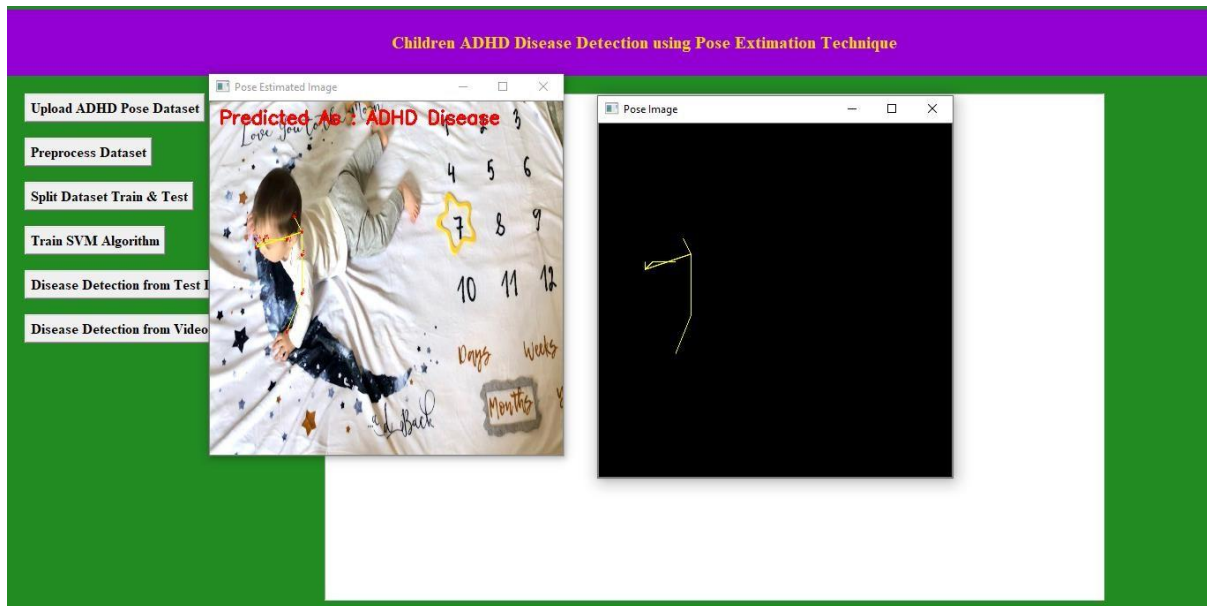


FIG 7: VIDEO UPLOADING PAGE

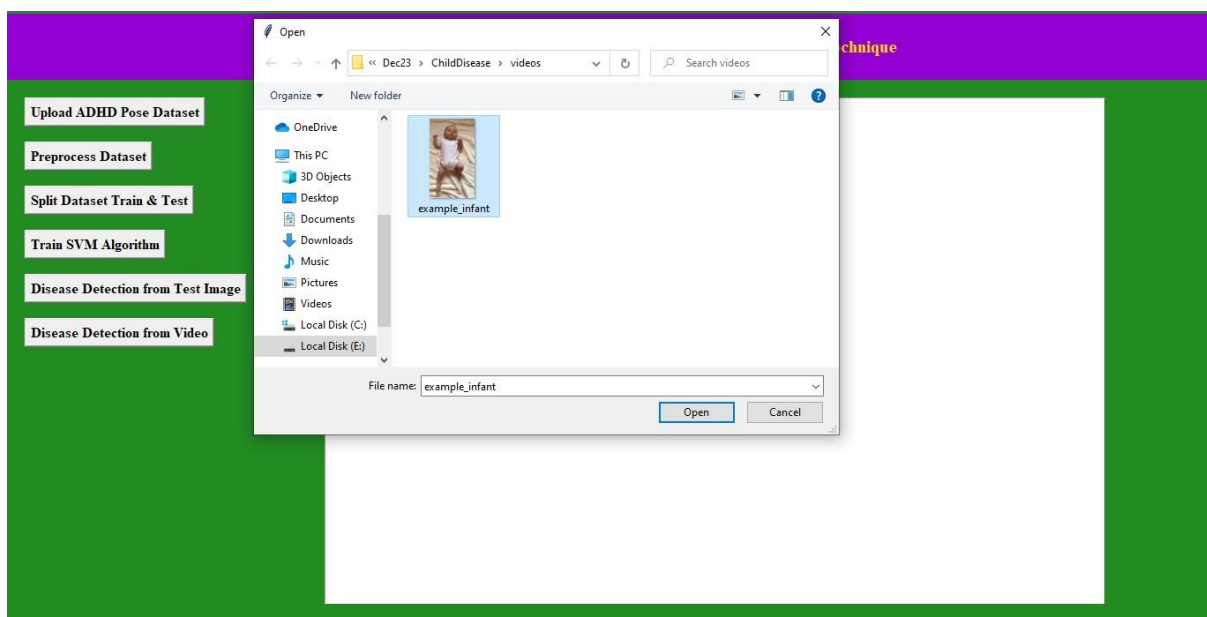


FIG
8: POSE ESTIMATION PAGE

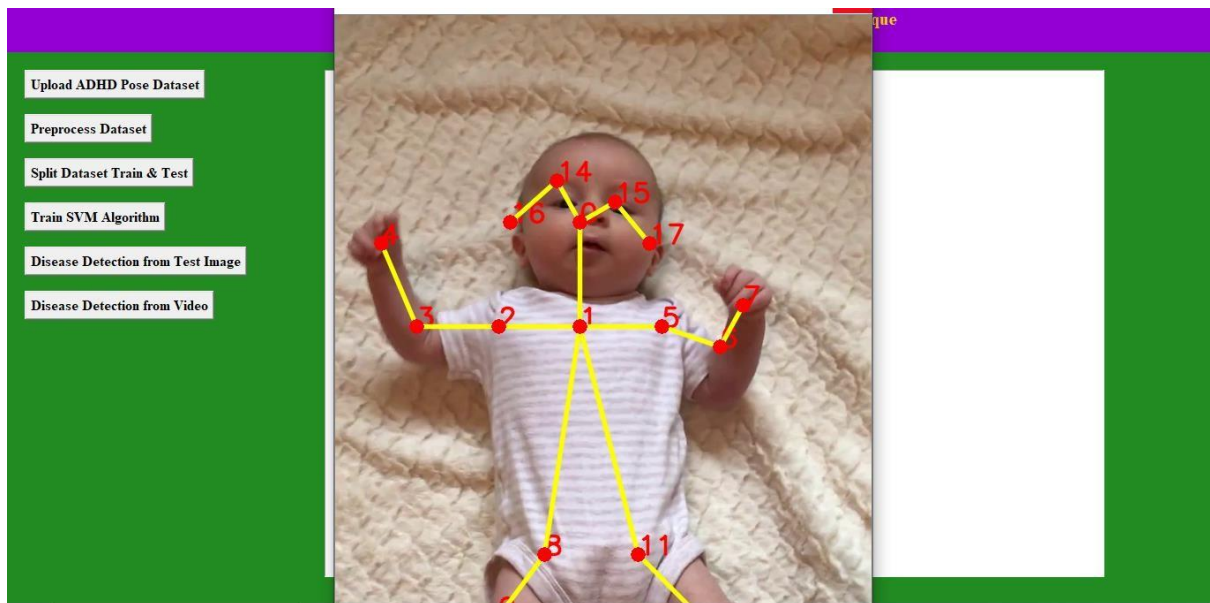
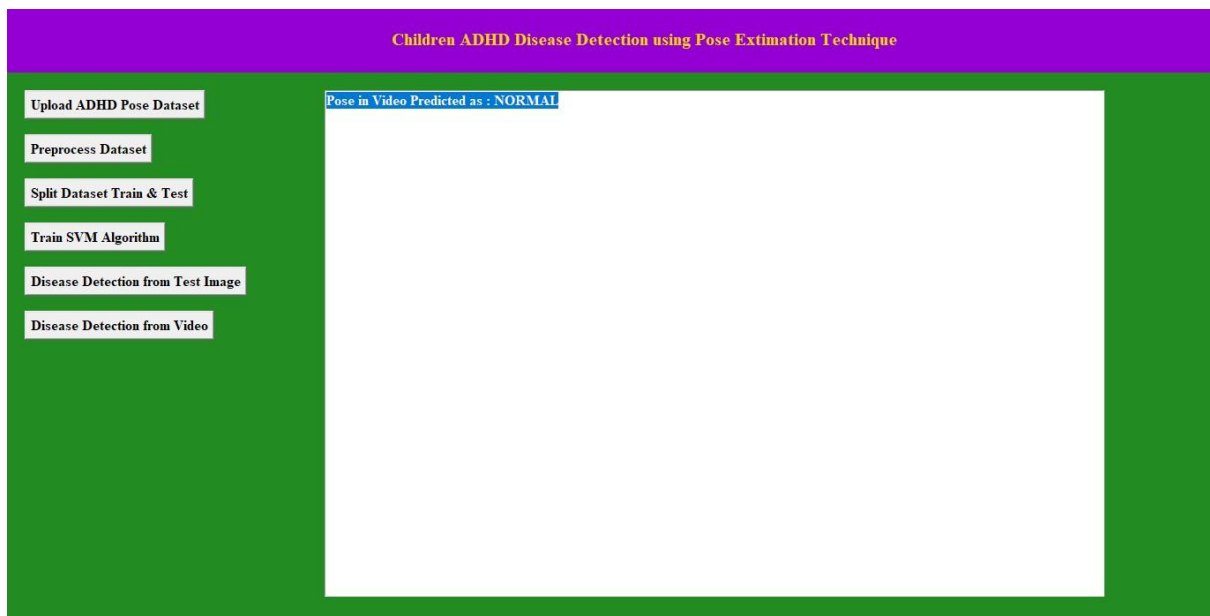


FIG 9: RESULT PAGE



CHAPTER-11

CONCLUSION

11.1 CONCLUSION

In conclusion, the project addresses the existing challenges in ADHD detection by introducing a machine learning-based approach. The proposed system, utilizing the SVM algorithm, offers a promising solution to automate and enhance the accuracy of ADHD detection based on children's pose estimation. By transitioning from manual monitoring to a more sophisticated and automated system, this project contributes to the advancement of ADHD diagnostic capabilities. The successful implementation of the proposed system has the potential to positively impact the field of neurodevelopmental disorder diagnosis, fostering earlier identification and intervention for improved patient outcomes.

11.2 FUTURE SCOPE

In the realm of ADHD detection in children, the utilization of pose estimation techniques harbors vast potential for future development. As technology advances, refining algorithms for more precise and reliable detection will be pivotal. Integration with wearable devices could facilitate continuous monitoring, offering invaluable insights into movement patterns for early intervention. Further advancements in machine learning models, bolstered by extensive clinical validation, hold promise for enhancing accuracy and reliability. Additionally, prioritizing the design of user-friendly interfaces and addressing ethical considerations surrounding data privacy and bias will be crucial for widespread acceptance and responsible implementation. Longitudinal studies tracking ADHD progression and intervention effectiveness will provide invaluable insights into the disorder's nature and treatment efficacy. The future scope for utilizing pose estimation technology in ADHD detection among children is expansive, offering a pathway toward more effective, accessible, and ethically sound diagnostic approaches.

CHAPTER-12

REFERENCES

1. R. Kale, "Detection of ADHD using Machine Learning Algorithms".
2. A. News, "Artificial Intelligence (AI) Improves MRI Detection of ADHD".
3. S. K. William, "A Robust Machine Learning-Based Framework for the Automated Detection of ADHD Using Pupillometric Biomarkers and Time Series Analysis".
4. A. D. a. D. N. Mladenka Vukojevi, "Early Detection and Recognition of Children with ADHD (Attention Deficit Hiperactivity Disorder) Symptoms".
5. J. Mitrpanont, B. Bousai, N. Soonthornchart, K. Tuanghirunvimon and T. Mitrpanont, "iCare-ADHD: A Mobile Application Prototype For Early Child Attention Deficit Hyperactivity Disorder", 2018 Seventh ICT International Student Project Conference (ICTISPC), pp. 1-4, 2018.
6. C. S and V. K, "An Approach to Measure and Improve the Cognitive Capability of ADHD Affected Children Through EEG Signals", 2018 IEEE 18th International Conference on Advanced Learning Technologies (ICALT), pp. 314-318, 2018.
7. X. Yufei, "Research on the Alternate Neurobiological Models of ADHD", 2020 International Conference on Public Health and Data Science (ICPHDS), pp. 355-359, 2020.
8. S. Ayoubipour, H. Hekmati and N. Sho'ouri, "Analysis of EOG Signals Related to ADHD and Healthy Children Using Wavelet Transform", 2020 27th National and 5th International Iranian Conference on Biomedical Engineering (ICBME), pp. 294-297, 2020.
9. D. Kuang and L. He, "Classification on ADHD with Deep Learning", 2014 International Conference on Cloud Computing and Big Data, pp. 27-32, 2014.
10. A. J. Hao, B. L. He and C. H. Yin, "Discrimination of ADHD children based on Deep Bayesian Network", 2015 IET International Conference on Biomedical Image and Signal Processing (ICBISP 2015), pp. 1-6, 2015.
11. B. Ahkami and F. Ghassemi, "Studying the Effects of Different Inter-Stimulus-Intervals in ADHD Adults While Performing CPT", 2018 25th National and 3rd International Iranian Conference on Biomedical Engineering (ICBME), pp. 1-5, 2018.

- 12.** L. Dery and O. Musicant, "ADHD Detection from Driving Patterns", 2018 IEEE International Conference on the Science of Electrical Engineering in Israel (ICSEE), pp. 1-4, 2018.
- 13.** E. Mangina, G. Chiazese and T. Hasegawa, "AHA: ADHD Augmented (Learning Environment)", 2018 IEEE International Conference on Teaching Assessment and Learning for Engineering (TALE), pp. 774-777, 2018.
- 14.** G. Yang, J. Liu, F. Xu and Z. Liu, "Sustained attention differences between children with ADHD and normal children based on auditory-visual stimulation", 2012 2nd International Conference on Consumer Electronics Communications and Networks (CECNet), pp. 2922-2924, 2012.
- 15.** V. Shah and S. Modi, "Comparative Analysis of Psychometric Prediction System", 2021 Smart Technologies Communication and Robotics (STCR), pp. 1-5, 2021.
- 16.** D. Yao, H. Sun, X. Guo, V. D. Calhoun, L. Sun and J. Sui, "ADHD Classification Within and Cross Cohort Using an Ensembled Feature Selection Framework", 2019 IEEE 16th International Symposium on Biomedical Imaging (ISBI 2019), pp. 1265-1269, 2019.
- 17.** V. Mehta and S. Modi, "Employee Attrition System Using Tree Based Ensemble Method", 2021 2nd International Conference on Communication Computing and Industry 4.0 (C2I4), pp. 1-4, 2021.
- 18.** S. Modi and M. H. Bohara, "Facial Emotion Recognition using Convolution Neural Network", 2021 5th International Conference on Intelligent Computing and Control Systems (ICICCS), pp. 1339-1344, 2021.
- 19.** P. Chatpreecha and S. Usanavasin, "Extracting Social Network Contents to Classify ADHD Types Based on Behavioral Symptoms and Activities", 2018 3rd International Conference on Computational Intelligence and Applications (ICCIA), pp. 255-259, 2018.
- 20.** N. Qiang, Q. Dong, Y. Sun, B. Ge and T. Liu, "Deep Variational Autoencoder for Modeling Functional Brain Networks and ADHD Identification", 2020 IEEE 17th International Symposium on Biomedical Imaging (ISBI), pp. 554-557, 2020.